A NUMERICAL METHOD FOR SOLVING THE EIKONAL EQAUATION ON SIMPLICIAL TESSELATIONS*

SACHIN NATESH†

Abstract. In this paper, we describe a method for solving the Eikonal equation on domains in $\mathbb{R}^2$ or $\mathbb{R}^3$ tesselated by triangles or tetrahedra. Our method generalizes to simplices in higher dimensions, owing to our choice of solution representation under the Jacobi family of orthogonal polynomials on the simplex. First, we describe a spectral decomposition for functions supported on the simplex in terms of their weighted L_2 expansion coefficients under the Jacobi basis. Computing this expansion requires an accurate and efficient quadrature, and we provide a novel optimization routine based on approximate joint-diagonalization of Jacobi matrices for generating such quadratures. Then, we consider differential operators acting on coefficients of a Jacobi expansion. Recent results due to Townsend, Oliver and Vasil enable the construction of sparse differential operators on the standard triangle, interoperating within the Jacobi polynomial family parameter space, and leading to low-storage, banded discrete differential operators. We generalize these results to the tetrahedra (laying the path for generalizing to $d > 3$), and use the resulting differential operators to construct the Eikonal operator on the simplex. Ultimately, we must solve a non-linear PDE, and working in coefficient space implies that our solution variables must be thought of as functions. Hence, we reformulate the Eikonal problem in terms of an infinite-dimensional L_2 mininization problem, which can be discretized with the aforementioned numerical quadrature. As with the README, this is more of a blog, but it's much easier to typeset math here.

Key words. multivariate orthogonal polynomials, spectral methods, partial differential equations, Eikonal equations, simplex

MSC codes. 68Q25, 68R10, 68U05

1. Global representations to local solutions of the Eikonal Equation. Haven't really looked into the viscosity perspective, but..I think I converged on the same idea for solving the PDE. Consider the standard simplex

$$\mathcal{T}^d = \{\boldsymbol{x} = (x_1, \cdots x_d) \in \mathbb{R}^d \mid x_i \geq 0,\ 1 - |\boldsymbol{x}|_1 \geq 0\}$$

The *minimal* time $u(\boldsymbol{x})$ to reach the boundary $\partial\mathcal{T}^d$ from a point $x \in \mathcal{T}^d$ is computable through solving the Eikonal equation on $\mathcal{T}^d$:

$$\begin{cases} \sum_{i=1}^d (\partial_{x_i} u)^2 = \left(\frac{1}{f}\right)^2, & x \in \mathcal{T}^d, \\ u = q, & x \in \partial\mathcal{T}^d, \end{cases} \tag{1.1}$$

where $f : \overline{\mathcal{T}^d} \to \mathbb{R}^+_\star$ is the speed in the medium, $q : \partial\mathcal{T}^d \to \mathbb{R}^+$ is the time to leave $\mathcal{T}^d$ from a point on $\partial\mathcal{T}^d$. We seek to represent u under an optimal, finite, weighted L_2 expansion, wherein

$$u(\boldsymbol{x}) \approx \sum_{j=1}^n u_j P_j(\boldsymbol{x}) = \boldsymbol{u} \cdot \boldsymbol{P}(\boldsymbol{x}),$$

and $\boldsymbol{u}$ is the vector in $\mathbb{R}^n$ of coefficients of u under the elements of the basis $\boldsymbol{P}(\boldsymbol{x}) : \mathbb{R}^d \to \mathbb{R}^n$. Here $\approx$ must be taken in the sense of weighted L_2 convergence of the series to u as $n \to \infty$.

1.1. Jacobi bases on the Triangle. Let us specialize for $d = 2$. By P_k^n, we denote the k^{th} Jacobi polynomial on the triangle of degree n with parameters (a, b, c). When the parameters must be specified, we will refer to the same polynomial by $P_{k,n}^{(a,b,c)}$. The weight function is

$$\begin{aligned} W^{(a,b,c)}(x, y) &= x^{a-\frac{1}{2}} y^{b-\frac{1}{2}} (1 - x - y)^{c-\frac{1}{2}}, \\ w_{(a,b,c)} &= \Big(\int_{\mathcal{T}^2_{\text{ref}}} W(x, y) dx dy \Big)^{-1} = \frac{\Gamma(|\kappa| + \frac{3}{2})}{\Gamma(a + \frac{1}{2})\Gamma(b + \frac{1}{2})\Gamma(c + \frac{1}{2})} \\ w^{(a,b,c)}(\boldsymbol{x}) &= w_{(a,b,c)} W^{(a,b,c)}(\boldsymbol{x}) \end{aligned} \tag{1.2}$$

with $a, b, c > -1/2$ the Jacobi parameters, and $|\kappa| = a + b + c$. The orthonormal polynomials are given in terms of the 1D Jacobi polynomials by

$$\begin{aligned} P_k^n(x, y) &= (h_{k,n})^{-1} P_{n-k}^{2k+b+c, a-1/2}(2x - 1)(1 - x)^k P_k^{c-1/2, b-1/2}\Big(\frac{2y}{1 - x} - 1\Big), \\ (h_{k,n})^2 &= \frac{w_{a,b,c}}{(2n + |\kappa| + \frac{1}{2})(2k + b + c)} \\ &\quad \times \frac{\Gamma(n + k + b + c + 1)\Gamma(n - k + a + \frac{1}{2})\Gamma(k + b + \frac{1}{2})\Gamma(k + c + \frac{1}{2})}{(n - k)! k! \Gamma(n + k + |\kappa| + \frac{1}{2})\Gamma(k + b + c)}, \end{aligned} \tag{1.3}$$

and they satisfy a mutual orthonormality condition under the weighted inner product:

$$\langle P_{j,m}^{(a,b,c)}, P_{k,n}^{(a,b,c)} \rangle_{w^{(a,b,c)}} = \delta_{jk}\delta_{mn}. \tag{1.4}$$

*Submitted to the editors DATE.

Funding: This work is partially supported by the National Science Foundation under Grant Number 2110886.

†Department of Applied Mathematics, University of Colorado, Boulder, CO

For the 1D Jacobi polynomials, we have convenient explicit evaluations of the polynomials at the endpoints (for reference, see the DLMF by NIST). On the triangle, we have.

(i) $P_k^n(0,y) = (h_{k,n})^{-1}(-1)^{n-k}\binom{n-k+a-1/2}{n-k}P_k^{c-1/2,b-1/2}\Big(2y-1\Big)$

(ii) $P_k^n(x,0) = (h_{k,n})^{-1}P_{n-k}^{2k+b+c,a-1/2}(2x-1)(1-x)^k(-1)^k\binom{k+b-1/2}{k}$

(iii) $P_k^n(x,1-x) = (h_{k,n})^{-1}P_{n-k}^{2k+b+c,a-1/2}(2x-1)(1-x)^k\binom{k+c-1/2}{k}$

Moreover, we must derive a limit relation for evaluating the polynomials at $x=1$, the bottom right corner of the reference triangle [1]. That is, we seek

$$\lim_{x\to 1} P_k^{n,(a,b,c)}(x,y) = (h_{k,n})^{-1}\binom{n+k+b+c}{n-k}\lim_{x\to 1}(1-x)^k P_k^{c-1/2,b-1/2}\Big(\frac{2y}{1-x}-1\Big),$$

where we used the endpoint formula for the Jacobi polynomials on the interval $[-1,1]$ to reduce the limit. We proceed by a dominating order argument. The term $(1-x)^k \to 0$ as $x\to 1$, while

$$\lim_{x\to 1} P_k^{c-1/2,b-1/2}\Big(\frac{2y}{1-x}-1\Big) \propto \lim_{x\to 1}\Big(\frac{2y}{1-x}-1\Big)^k$$

is indeterminate. However, the combined limit can be easily solved as

$$\lim_{x\to 1}(1-x)^k\Big(\frac{2y}{1-x}-1\Big)^k = (2y)^k,$$

and $(1-x)^k\Big(\frac{2y}{1-x}-1\Big)^l \to 0$ as $x\to 1$ for any $l<k$. Hence, the leading order coefficient of $P_k^{c-1/2,b-1/2}\Big(\frac{2y}{1-x}-1\Big)$ is the needed proportionality constant (see DLMF), so that

$$\lim_{x\to 1} P_k^{n,(a,b,c)}(x,y) = (h_{k,n})^{-1}\binom{n+k+b+c}{n-k}\frac{(k+c+b)_k}{2^k k!}(2y)^k, \tag{1.5}$$

where $(a)_n$ is the Pochhammer symbol.

A given basis polynomial is not only orthonormal to those with different total degree, but also to those others within the same-degree homogeneous subspace. In other words, $P_k^n,\ 0\le k\le n$ forms a basis for the space of homogeneous polynomials of degree n. By considering ordered collections of these subspaces (where ordering is in terms of the degree n of the space), instead of individual polynomials, we can more elegantly reformulate the Eikonal problem into a weak-type form. Let

$$\mathbb{P}_n(x_1,x_2) = (P_0^n(x_1,x_2),\cdots P_n^n(x_1,x_2))^T,$$

and let

$$\boldsymbol{P} = (\mathbb{P}_0^T,\mathbb{P}_1^T,\cdots,\mathbb{P}_{n-1}^T)^T. \tag{1.6}$$

The above notation amounts to a vectorization of the basis for the general space of polynomials in two variables Π_{n-1}^2 defined of $\mathcal{T}^2$. Now, consider the $(n-1)$ order Jacobi polynomial expansion of the solution u under the parameter family (a,b,c). For $\boldsymbol{x}\in\mathcal{T}^d$, and $N=\dim\Pi_{n-1}^2$ coefficients $\boldsymbol{u}\in\mathbb{R}^N$, we can write it as

$$u(\boldsymbol{x}) = \boldsymbol{P}^{(a,b,c)}(\boldsymbol{x})^T\boldsymbol{u}. \tag{1.7}$$

There exists sparse discrete operators which map $\boldsymbol{u}$ to the coefficients of the derivative of u under a modified basis, as well as those which promote the basis parameters without taking derivatives:

$$\begin{aligned}\partial_{x_1}u(\boldsymbol{x}) &= \boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T\mathcal{K}_{(a+1,b,c+1)}^{(a+1,b+1,c+1)}\mathcal{D}_{x_1}\boldsymbol{u}\\ \partial_{x_2}u(\boldsymbol{y}) &= \boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T\mathcal{K}_{(a,b+1,c+1)}^{(a+1,b+1,c+1)}\mathcal{D}_{x_2}\boldsymbol{u}\end{aligned} \tag{1.8}$$

Inserting this into the fully constrained Eikonal equation gives

$$\begin{aligned}&\Big(\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T\mathcal{K}_{(a,b,c)}^{(a+1,b+1,c+1)}\boldsymbol{c}_{\frac{1}{f}}\Big)^2 = \\ &\big(\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T\mathcal{K}_{(a+1,b,c+1)}^{(a+1,b+1,c+1)}\mathcal{D}_{x_1}\boldsymbol{u}\big)^2 + \\ &\big(\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T\mathcal{K}_{(a,b+1,c+1)}^{(a+1,b+1,c+1)}\mathcal{D}_{x_2}\boldsymbol{u}\big)^2 =\end{aligned} \tag{1.9}$$

$$\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T\big(\tilde{\mathcal{D}}_{x_1}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}_{x_1}^T + \tilde{\mathcal{D}}_{x_2}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}_{x_2}^T\big)\boldsymbol{P}^{(a+1,b,c+1)}(\boldsymbol{x}), \tag{1.10}$$

where

$$\tilde{\mathcal{D}}_{x_1} = \mathcal{K}_{(a+1,b,c+1)}^{(a+1,b+1,c+1)}\mathcal{D}_{x_1},\quad \tilde{\mathcal{D}}_{x_2} = \mathcal{K}_{(a,b+1,c+1)}^{(a+1,b+1,c+1)}\mathcal{D}_{x_2}, \tag{1.11}$$

[1] This is necessary for evaluating the Vandermonde matrix at such points, as they may occur during descent iterations for the quadrature problem

and, the coefficients of $(1/f)$ are represented as $\boldsymbol{c}_{\frac{1}{f}}$ but promoted to the $(a+1,b+1,c+1)$ Jacobi basis. We let $\tilde{\cdot}$ indicate promotion *to* $(a+1,b+1,c+1)$ from any other parameter set. The above equality holds for any point $\boldsymbol{x} \in \mathcal{T}^2$. So, for a solution $\boldsymbol{u}$, the coefficient operation must be such that:

$$\left(\tilde{\mathcal{D}}_{x_1}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}_{x_1}^T + \tilde{\mathcal{D}}_{x_2}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}_{x_2}^T\right) = \tilde{\boldsymbol{c}}_{\frac{1}{f}}\tilde{\boldsymbol{c}}_{\frac{1}{f}}^T. \tag{1.12}$$

We seek a coefficient solution $\boldsymbol{u}$ which represents the solution *globally* in $\mathcal{T}^2$. However, polynomials are differentiable, and some of the problems we are interested in yield non-differentiable solutions. This is true even in the most basic case of $f \equiv 1$, in which u is the signed distance function from $\boldsymbol{x}$ to $\partial\mathcal{T}^2$, depicted below:

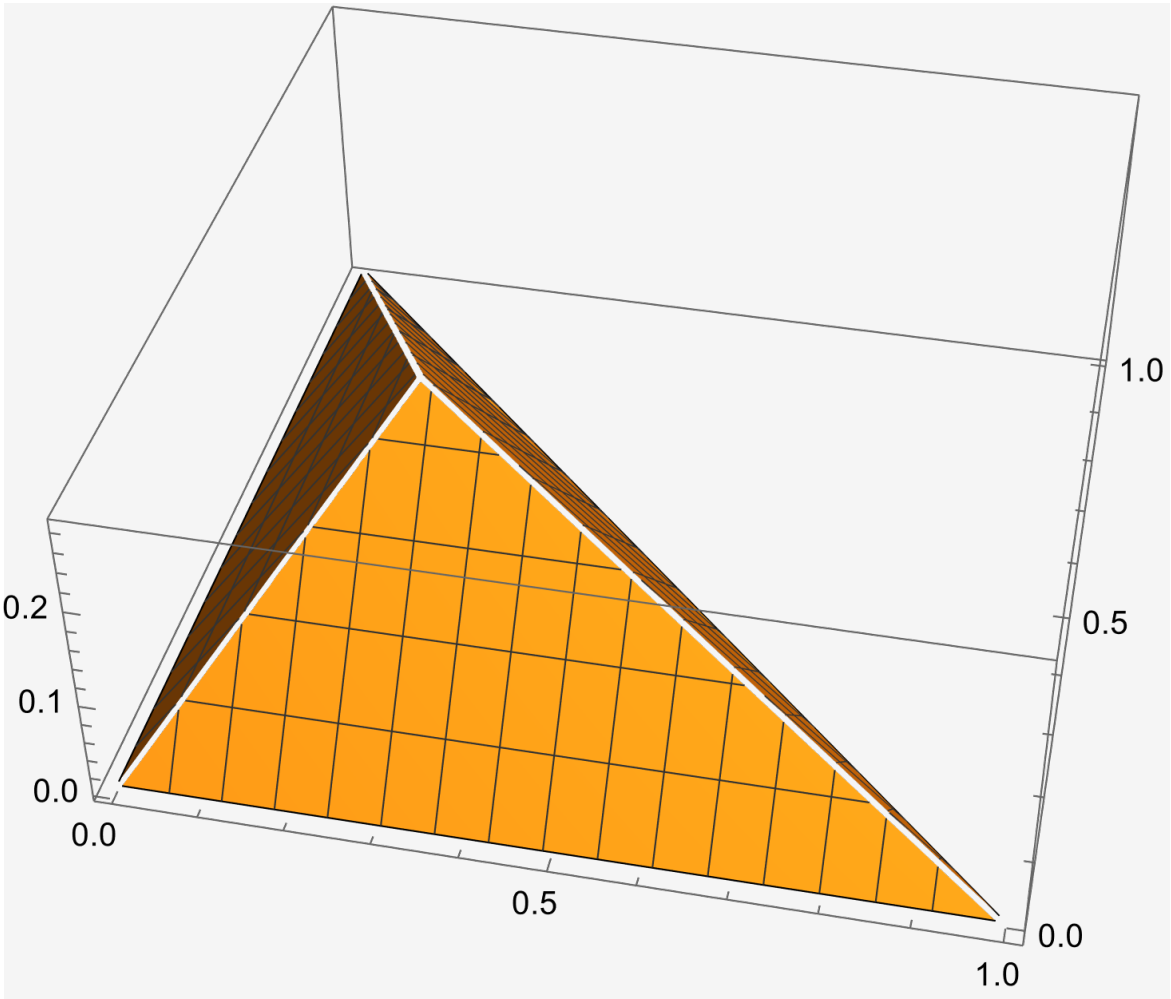

FIG. 1. *A simple calculation shows that the minimum distance from* $\boldsymbol{x} \in \mathcal{T}^2$ *to the boundary* $\partial\mathcal{T}^2$ *is given by the minimum of* $\left\{x_1, x_2, \left|\left|\boldsymbol{x} - \begin{bmatrix}(x_1 - x_2 + 1)/2\\ (x_2 - x_1 + 1)/2\end{bmatrix}\right|\right|_2\right\}$

We cannot hope to recover a solution $\boldsymbol{u}$ which satisfies (1.12) to machine precision, as that would imply the global solution has zero residual *pointwise* everywhere. Hence, we must return to (1.9), and consider solving it weakly, in the sense of:

$$\begin{aligned}
&\int_{\mathcal{T}^2} w^{(a+1,b+1,c+1)}(\boldsymbol{x})\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T\tilde{\boldsymbol{c}}_{\frac{1}{f}}\tilde{\boldsymbol{c}}_{\frac{1}{f}}^T\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})d\boldsymbol{x} \approx ||1/f||^2_{w_{(a+1,b+1,c+1)}}\\
&\approx \int_{\mathcal{T}^2} w^{(a+1,b+1,c+1)}(\boldsymbol{x})\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})^T\left(\tilde{\mathcal{D}}_{x_1}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}_{x_1}^T + \tilde{\mathcal{D}}_{x_2}\boldsymbol{u}\boldsymbol{u}^T\tilde{\mathcal{D}}_{x_2}^T\right)\boldsymbol{P}^{(a+1,b+1,c+1)}(\boldsymbol{x})d\boldsymbol{x}\\
&\approx ||\partial_x u||^2_{w^{(a+1,b+1,c+1)}} + ||\partial_y u||^2_{w^{(a+1,b+1,c+1)}}.
\end{aligned} \tag{1.13}$$

Here, the norm is viewed as that of a function in $L^2(\mathcal{T}^2, w^{(a+1,b+1,c+1)})$. Parsing this equation out a bit further (condensing sub/super

scripts), we have

$$
\begin{aligned}
&\int_{\mathcal{T}^2} \boldsymbol{P}(\boldsymbol{x})^T(\boldsymbol{u_x}\boldsymbol{u_x}^T + \boldsymbol{u_y}\boldsymbol{u_y}^T)w(\boldsymbol{x})\boldsymbol{P}(\boldsymbol{x})d\boldsymbol{x} \quad (\textbf{Only } x \textbf{ part handled below})\\
&= \int_{\mathcal{T}^2} \boldsymbol{P}(\boldsymbol{x})^T \begin{bmatrix} u_{x,1}^2 & u_{x,1}u_{x,2} & \cdots & u_{x,1}u_{x,N} \\ u_{x,2}u_{x,1} & u_{x,2}^2 & \cdots & u_{x,2}u_{x,N} \\ \vdots & \ddots & \ddots & \\ u_{x,N}u_{x,1} & u_{x,N}u_{x,2} & \cdots & u_{x,N}^2 \end{bmatrix} \boldsymbol{P}(\boldsymbol{x})w(\boldsymbol{x})d\boldsymbol{x}\\
&= \int_{\mathcal{T}^2} \begin{bmatrix} P_1(\boldsymbol{x}) & P_2(\boldsymbol{x}) & \cdots & P_N(\boldsymbol{x}) \end{bmatrix} \begin{bmatrix} u_{x,1}^2 & u_{x,1}u_{x,2} & \cdots & u_{x,1}u_{x,N} \\ u_{x,2}u_{x,1} & u_{x,2}^2 & \cdots & u_{x,2}u_{x,N} \\ \vdots & \ddots & \ddots & \\ u_{x,N}u_{x,1} & u_{x,N}u_{x,2} & \cdots & u_{x,N}^2 \end{bmatrix} \begin{bmatrix} P_1(\boldsymbol{x}) \\ P_2(\boldsymbol{x}) \\ \vdots \\ P_N(\boldsymbol{x}) \end{bmatrix} w(\boldsymbol{x})d\boldsymbol{x}\\
&= \int_{\mathcal{T}^2} \begin{bmatrix} P_1(\boldsymbol{x}) & P_2(\boldsymbol{x}) & \cdots & P_N(\boldsymbol{x}) \end{bmatrix} \begin{bmatrix} u_{x,1}\sum_{j=1}^N u_{x,j}P_j(\boldsymbol{x}) \\ u_{x,2}\sum_{j=1}^N u_{x,j}P_j(\boldsymbol{x}) \\ \vdots \\ u_{x,N}\sum_{j=1}^N u_{x,j}P_j(\boldsymbol{x}) \end{bmatrix} w(\boldsymbol{x})d\boldsymbol{x}\\
&= \int_{\mathcal{T}^2} w(\boldsymbol{x}) \sum_{k=1}^N P_k(\boldsymbol{x})u_{x,k} \sum_{j=1}^N u_{x,j}P_j(\boldsymbol{x})d\boldsymbol{x} = \int_{\mathcal{T}^2} w(\boldsymbol{x}) \sum_{k=1}^N \sum_{j=1}^N u_{x,k}u_{x,j}P_k(\boldsymbol{x})P_j(\boldsymbol{x})d\boldsymbol{x}\\
&= \int_{\mathcal{T}^2} \sum_{k=1}^N u_{x,k}^2 P_k^2(\boldsymbol{x})w(\boldsymbol{x})d\boldsymbol{x} = \sum_{k=1}^N u_{x,k}^2 \int_{\mathcal{T}^2} P_k^2(\boldsymbol{x})dw(\boldsymbol{x}) \quad (\textbf{ now we add back in } y \textbf{ part })\\
&= \sum_{k=1}^N u_{x,k}^2 + \sum_{k=1}^N u_{y,k}^2\\
&= \boldsymbol{u}^T(\tilde{\mathcal{D}}_x^T\tilde{\mathcal{D}}_x + \tilde{\mathcal{D}}_y^T\tilde{\mathcal{D}}_y)\boldsymbol{u} = \tilde{\boldsymbol{c}}_{\frac{1}{f}}^T\tilde{\boldsymbol{c}}_{\frac{1}{f}},
\end{aligned}
$$

where we've used the mutual orthonormality relations, which persist even within a subspace of homogeneous polynomials of degree m, to eliminate all off-diagonal terms in the sum. Note, not all orthonormal bases are created equal, and we normally would have to retain many outer product terms in the sum because orthonormality between vectors of the same subspace is not a given.

Thus, we want to solve the following quadratic program with equality constraints:

$$\min_{\boldsymbol{u}\in\mathbb{R}^N} F(\boldsymbol{u}) = \boldsymbol{u}^T G\boldsymbol{u} - \tilde{\boldsymbol{c}}_{\frac{1}{f}}^T \tilde{\boldsymbol{c}}_{\frac{1}{f}} \tag{1.14}$$
$$\text{subject to} \quad \boldsymbol{u}\cdot\boldsymbol{P}(\boldsymbol{x}) = 0, \quad \boldsymbol{x}\in\partial\Omega,$$

where $G = \tilde{\mathcal{D}}_x^T\tilde{\mathcal{D}}_x + \tilde{\mathcal{D}}_y^T\tilde{\mathcal{D}}_y \in \mathbb{R}^{N\times N}$ is symmetric and positive semi-definite [2]. The gradient and Hessian of F are given by

$$\nabla_{\boldsymbol{u}} F(\boldsymbol{u}) = 2G\boldsymbol{u}, \tag{1.15}$$
$$\nabla^2_{\boldsymbol{u}} F(\boldsymbol{u}) = 2G, \tag{1.16}$$

and we still need to consider the constraint equation. We take the approach of eliminating the constraints by solving a the minimization problem on auxiliary variables. That is, suppose $\boldsymbol{P}(\boldsymbol{x}) \in \mathbb{R}^{M\times N}$, $M > N$, has rank p. We can compute the *full SVD* of $\boldsymbol{P}(\boldsymbol{x}) = U\Sigma V^T$. Then, the last $n-p$ rows of V^T form a basis for the $\mathcal{N}(\boldsymbol{P}(\boldsymbol{x}))$. That is, $V^T = \begin{bmatrix} V_p^T \\ V_{n-p}^T \end{bmatrix}$, and any solution $\boldsymbol{u}$ satisfying the equality constraints can be expressed as

$$\boldsymbol{u} = V_{n-p}\boldsymbol{u}_{\text{eq}}, \tag{1.17}$$

Hence, we can solve

$$\min_{\boldsymbol{u}_{\text{eq}}\in\mathbb{R}^N} F(\boldsymbol{u}_{\text{eq}}) = \boldsymbol{u}_{\text{eq}}^T V_{n-p}^T G V_{n-p}\boldsymbol{u}_{\text{eq}} - \tilde{\boldsymbol{c}}_{\frac{1}{f}}^T \tilde{\boldsymbol{c}}_{\frac{1}{f}} \tag{1.18}$$

and finally find $\boldsymbol{u}$ by computing as in (1.17). Note, the gradient and Hessian of the auxiliary problem are given by

$$\nabla_{\boldsymbol{u}_{\text{eq}}} F(\boldsymbol{u}_{\text{eq}}) = 2V_{n-p}^T G V_{n-p}\boldsymbol{u}_{\text{eq}}, \tag{1.19}$$
$$\nabla^2_{\boldsymbol{u}} F(\boldsymbol{u}) = 2V_{n-p}^T G V_{n-p}. \tag{1.20}$$

Note, we can find p by the number of singular values above some small threshold that we choose (near $\varepsilon_{\text{mach}}$) so that it represents numerical rank.

Remark 1.1. There is a ton of material in chapter 12 and 16 of the Nocedal/Wright optimization book. I believe I can prove properties on minimizers to (1.14) (though unnecessary) and maybe use a direct method therein. There still remains the problem of initialization..perhaps we just initialize from the null space, or solve a linearized problem?

Remark 1.2. By making use of a weighted basis $\tilde{\boldsymbol{P}}(\boldsymbol{x})$, and expressing our solution coefficients and differential operators in terms of this weighted basis, we can alternatively eliminate the constraint equation and solve a completely unconstrained quadratic optimization problem I think I need to do some more symbolic computation to figure out the form of the RHS and/or additional conditions u must first satisfy so that we solve the same problem. I implemented that weighted basis, but I don't think I will get things to work properly, or rather, I will not continue trying.

Remark 1.3. It turns out that this squared version of the Eikonal problem is not coercive. That is, there is no associated coercive bilinear form, which we need if we want to initialize the method with something other than the true solution! The elegance of a polynomial identity in coefficient space is, alas, forgone in practice. Instead, we turn to a regularized version of the un-squared Eikonal problem, and try our hand at a modal finite element method.

[2] Note, $\forall \boldsymbol{x}\in\mathbb{R}^{n^+_\star}, \boldsymbol{x}^T G\boldsymbol{x} = \boldsymbol{x}^T\tilde{\mathcal{D}}_x^T\tilde{\mathcal{D}}_x\boldsymbol{x} + \boldsymbol{x}^T\tilde{\mathcal{D}}_y^T\tilde{\mathcal{D}}_y\boldsymbol{x} = ||\tilde{\mathcal{D}}_x\boldsymbol{x}||^2 + ||\tilde{\mathcal{D}}_y\boldsymbol{x}||^2 \geq 0$. We have equality only for any $\boldsymbol{x} = \alpha\boldsymbol{e}_1$, as derivatives of the constant polynomial are 0.

1.2. Regularized Eikonal Problem. Here, we consider the regularized Eikonal problem

$$|\nabla u| - \left(\frac{1}{f}\right) = \xi \nabla^2 u, \quad x \in \Omega \tag{1.21}$$
$$u = g, \quad x \in \partial\Omega$$

which we discretize with a *modal* finite-element method. That is, let $\Omega = \mathcal{T} = \bigcup_{k=1}^{N_T} T_k$ be tesselated by triangles T_k. Let A_k map points $\boldsymbol{x} \in T_k$ to points $\boldsymbol{r} \in T_{\text{ref}}$ the reference triangle. For $\boldsymbol{x} \in T_k$, we represent the solution

$$u^k(\boldsymbol{x}) = \sum_{j=1}^{M} u_j^k \phi_j(A_k^{-1}\boldsymbol{x}),$$

where u_j^k are the Jacobi polynomial coefficients associated to Jacobi basis function ϕ_j. Each triangle uses the same basis functions with mapped argument, and we stack the M unknown coefficients for each triangle as

$$\boldsymbol{U} = \begin{bmatrix} \boldsymbol{u}^1 \\ \vdots \\ \boldsymbol{u}^{N_T} \end{bmatrix} \in \mathbb{R}^{MN_T}.$$

We enforce Dirichlet conditions on all boundary edges, and inter-element continuity on all interior edges. Let

$$\mathcal{E}_{\text{dirch}} = \{\partial T_i \cap \partial\Omega\} = \{\varepsilon^i_{\text{dirch}}\}, \quad \mathcal{E}_{\text{int}} = \{\varepsilon^i_{\text{int}}\}$$

be the set of boundary and interior edges in the tesselation $\mathcal{T}$. We have the associated index maps between T_j and boundary edge index i, and interior edge index i with triangles j, k:

$$b(i) = \begin{cases} j, & \varepsilon^i_{\text{dirch}} \cap \partial T_j \neq \emptyset \\ \emptyset, & \text{else} \end{cases}, \quad t(i) = (t_1(i), t_2(i)) = \Big\{(j,k), \quad (\varepsilon^i_{\text{int}} \cap \partial T_j \neq \emptyset) \wedge (\varepsilon^i_{\text{int}} \cap \partial T_k \neq \emptyset)\ .$$

Note, the map t is always non-empty, as every interior edge is shared by two triangles (probably have to reformulate notation for this map). Now, we proceed with the normal weak formulation of (1.21), in which we evaluate the entries of the block diagonal stiffness matrix, nonlinear functional and load vector, $\mathcal{K} \in \mathbb{R}^{MN_T \times MN_T}$ (each of the N_T blocks is $M \times M$), $\boldsymbol{N}(\boldsymbol{U}) \in \mathbb{R}^{MN_T}, \boldsymbol{F} \in \mathbb{R}^{MN_T}$, with a Gaussian-like quadrature rule consisting of abscissa and weights $\{\boldsymbol{r}_j, w_j\}$ (see Section 2) on T_{ref}. For triangle T_k, we have for $i, j = 1, \cdots M$, $k = 1, \cdots N_T$,

$$\mathcal{K}^k_{ij} = \int_{T_{\text{ref}}} (A_k^{-1}\nabla\phi_i) \cdot (A_k^{-1}\nabla\phi_j)|A_k| d\boldsymbol{r}, \quad \mathcal{K} = \begin{bmatrix} \mathcal{K}^1 & & \\ & \ddots & \\ & & \mathcal{K}^{N_T} \end{bmatrix} \tag{1.22}$$

$$\mathcal{N}(\boldsymbol{u}^k)_i = \int_{T_{\text{ref}}} \phi_i |A_k| \sqrt{\sum_{l=1}^{M}\sum_{p=1}^{M} u_l^k u_p^k (A_k^{-1}\nabla\phi_l) \cdot (A_k^{-1}\nabla\phi_p)} d\boldsymbol{r}, \quad \boldsymbol{N}(\boldsymbol{U}) = \begin{bmatrix} \mathcal{N}(\boldsymbol{u}^1)_1 \\ \vdots \\ \mathcal{N}(\boldsymbol{u}^{N_T})_M \end{bmatrix} \tag{1.23}$$

$$\mathcal{F}^k_i = \int_{T_{\text{ref}}} \frac{\phi_i}{f} |A_k| d\boldsymbol{r}, \quad \boldsymbol{F} = \begin{bmatrix} \mathcal{F}^1_1 \\ \vdots \\ \mathcal{F}^{N_T}_M \end{bmatrix} \tag{1.24}$$

We use the derivative maps, as in (1.8), to compute the gradients of the basis functions appearing in (1.22)-(1.23). That is,

$$\partial_x \phi_j^{(a,b,c)}(\boldsymbol{r}) = \partial_x \boldsymbol{\phi}^{(a,b,c)}(\boldsymbol{r})^T \boldsymbol{e}_j = \boldsymbol{\phi}^{a+1,b,c+1}(\boldsymbol{r}) \mathcal{D}_x \boldsymbol{e}_j,$$

and similar for $\partial_y \phi_j^{(a,b,c)}$. At this point, we should note that our basis functions to not encode the boundary conditions, and are non-conforming between elements. Hence, we enforce Dirichlet boundary conditions and inter-element continuity using Lagrange multipliers. We can do this in two ways - weakly or pointwise. In the weak constraint enforcement, we seek a solution $\boldsymbol{U}$ which lies in the null-space of a Lagrange multiplier function space, defined as the span of the edge-restricted basis functions. In the point-wise enforcement, we seek a solution which lies in the null-space of a finite-dimensional Lagrange multiplier vector space, which constrains the solution at finitely many points. We take the latter approach. In fact, if we use the same quadrature points on the edges for the strong enforcement as in the quadrature rules for the weak enforcement, numerical satisfaction of the point-wise constraints implies that we satisfy the weak constraints as well.

Consider n_{leg} Legendre points $\{r_j\}$ in $[0, 1]$. Each edge of the reference triangle can be parameterized with this set of points.

- The left edge is $\{\boldsymbol{x}^l\} = \{(0, r_j)\}$
- The bottom edge is $\{\boldsymbol{x}^b\} = \{(r_j, 0)\}$
- The hypotenuse edge is $\{\boldsymbol{x}^h\} = \{(r_j, 1 - r_j)\}$

Say Dirichlet edge $\varepsilon^i_{\text{dirch}}$ is discretized with Legendre points $\boldsymbol{x}^e$ (it could be any of the edges in the reference). Then, we want

$$\boldsymbol{\phi}(\boldsymbol{x}^e)^T \boldsymbol{u}^{b(i)} = 0 \text{ (or } g(\boldsymbol{x}^e) \text{ if condition is non-homogeneous).} \tag{1.25}$$

Similarly, for shared edge $\varepsilon^i_{\text{int}}$, we require

$$\boldsymbol{\phi}(\boldsymbol{x}^e)^T \boldsymbol{u}^{t_1(i)} - \boldsymbol{\phi}(\boldsymbol{x}^e)^T \boldsymbol{u}^{t_2(i)} = 0. \tag{1.26}$$

In the actual assembly procedure, care must be taken to ensure these evaluations occur as written. That is, a Dirichlet edge in physical space may be flipped in the reference triangle. A shared edge in physical space may belong to different edges in the sharing triangles when mapped to reference space, and the parametrization may be flipped as well. Hence, we use edge alignment detection procedures to ensure the basis evaluations are aligned. For Dirichlet edges, this is as simple as mapping the edge vertices to the reference triangle, and checking if the mapped vertices are flipped (eg. edge is mapped to (1,0)-(0,0) is bottom flipped). If they are, we flip the rows of $\boldsymbol{\phi}(\boldsymbol{x}^e)^T$. For shared edges, we detect if one edge is flipped relative to the other by mapping the quadrature points using either triangles' A_j, A_k incidence matrices, and doing the same for the flipped quadrature points for one of the triangles. If the norm of the difference in physical space between the flipped quadrature points of T_k and the un-flipped quadrature points of T_j is $\mathcal{O}(\varepsilon_{\text{mach}})$, then the basis of T_k must be flipped.

Ultimately, we assemble two matrices to enforce these constraints. Let $n_{\text{bnd}} = |\mathcal{E}_{\text{dirch}}|$ and $n_{\text{int}} = |\mathcal{E}_{\text{int}}|$. Then, we define

$$B^{\text{dirch}} \in \mathbb{R}^{n_{\text{bnd}} n_{\text{leg}} \times MN_T}, \quad B^{\text{int}} \in \mathbb{R}^{n_{\text{int}} n_{\text{leg}} \times MN_T}$$

which are both block-sparse. B^{dirch} has n_{bnd} row blocks, one for each boundary edge, and each is of size $n_{\text{leg}} \times MN_T$ with only $n_{\text{leg}} \times M$ non zeros. B^{int} has n_{int} row blocks, on for each interior edge, and each is of size $n_{\text{int}} \times MN_T$, with two separate sub-blocks of size $n_{\text{leg}} \times M$. The specific structure of these matrices depends entirely on the geometry and tesselation, though we can specify how a given row block of either matrix is computed. For boundary edge i,

$$B^{\text{dirch},i}_{k,(b(i)-1)M+1:b(i)M} = \boldsymbol{\phi}(x^e_k)^T,$$

while for shared edge i,

$$B^{\text{int},i}_{k,(t_1(i)-1)M+1:t_1(i)M} = \boldsymbol{\phi}(x^e_k)^T, \quad B^{\text{int},i}_{k,(t_2(i)-1)M+1:t_2(i)M} = -\boldsymbol{\phi}(x^e_k)^T$$

For clarity, we visualize the structure of these matrices for a unit-square domain tesselated with 18 triangles and an order 10 discretization.

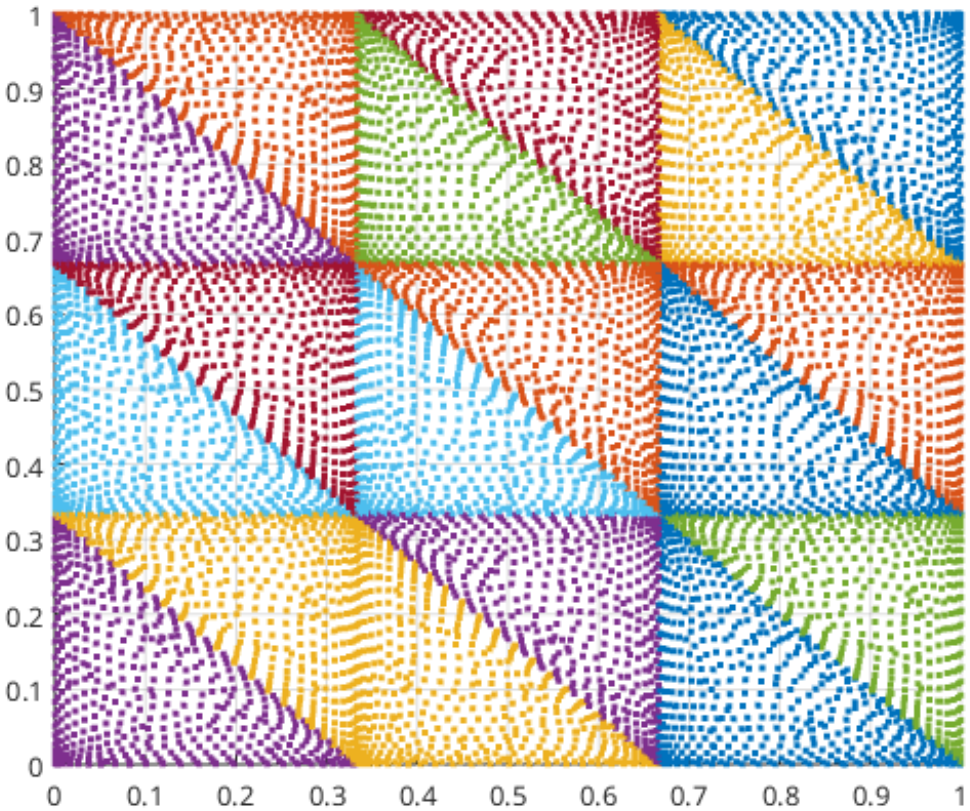

FIG. 2. $[0,1]^2$ *tesselated with 18 triangles, showing mapped quadrature points in each triangle.*

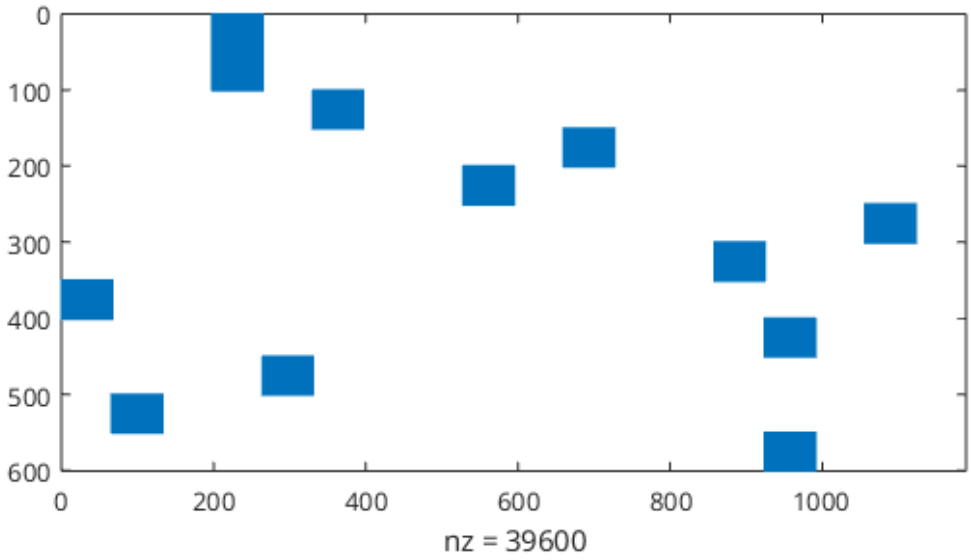

FIG. 3. *Structure of* B^{dirch}. $M = 66, N_T = 18, n_{bnd} = 12, n_{leg} = 50$

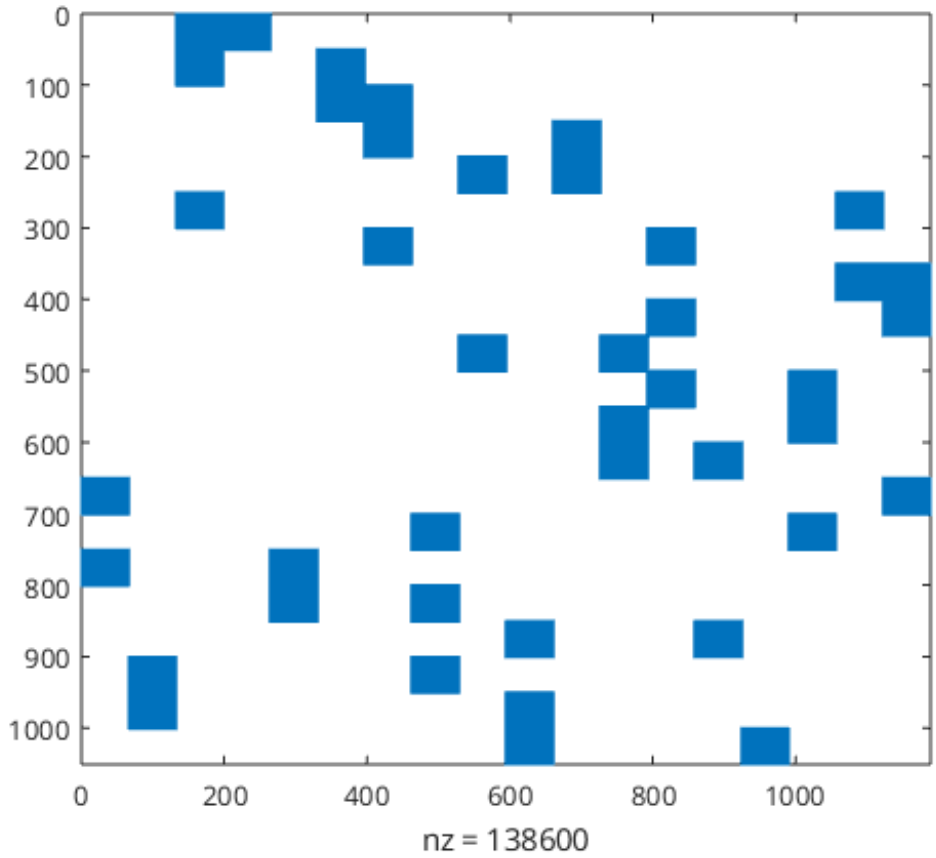

FIG. 4. *Structure of* B^{int}. $M = 66, N_T = 18, n_{int} = 21, n_{leg} = 50$

The structure we described earlier is evident from these figures. In particular, each block row of B^{int} has two non-zero blocks of size $n_{\text{leg}} \times M$, and each block row of B^{dirch} has only one non-zero block of size $n_{\text{leg}} \times M$. Also note, the bottom-left and upper-right triangles in the mesh both have two boundary edges, and these manifest as block columns in B^{dirch} with two non-zero blocks of size $n_{\text{leg}} \times M$ (see block columns 4 and 15).

At this point, we have not addressed the rank deficiency of the combined matrix

$$B = \begin{bmatrix} B^{\text{int}} \\ B^{\text{dirch}} \end{bmatrix}.$$

There are row redundancies in the constraint equations, both within the Dirichlet and inter-element matrices, as well as between them. To eliminate these redundancies and arrive at a full-rank constraint system, we employ an interpolatory decomposition to select only the independent rows of B, described below.

- Use a rank-revealing `QR` factorization to get pivot information, providing the independent rows of B. Arrange the pivot information as a vector P so that $B^T(:, P) = QR$. Let $n_\lambda = \texttt{rank}(B)$.
- Construct $B_{\text{id}} = B(P(1 : n_\lambda, :))$. This new matrix has full rank.
- In the case of non-zero Dirichlet conditions g, we must carefully select the correct elements of the assembled vector G, according to the permutation vector P, as in

$$\boldsymbol{G}_{\text{bc}} = \begin{bmatrix} \boldsymbol{O}_{n_{\text{leg}} n_{\text{int}}} \\ G \end{bmatrix}, \quad \boldsymbol{G}_{\text{bc}} \to \boldsymbol{G}_{\text{bc}}(P(1 : n_\lambda))$$

Now, we can specify the fully discretized variational formulation. We seek $\boldsymbol{U} \in \mathbb{R}^{MN_T}, \boldsymbol{\Lambda} \in \mathbb{R}^{n_\lambda}$ such that

$$\begin{aligned} \boldsymbol{N}(\boldsymbol{U}) + \xi \mathcal{K} \boldsymbol{U} - \boldsymbol{F} + B_{\text{id}}^T \boldsymbol{\Lambda} &= 0 \\ B_{\text{id}} \boldsymbol{U} &= \boldsymbol{G}_{\text{bc}} \ (\equiv 0 \text{ for the typical problem}) \end{aligned} \tag{1.27}$$

Here, we have a constrained non-linear system of equations, and there are many ways in which we can attempt to solve it. We define the residual of the system as

$$\boldsymbol{R}(\boldsymbol{U}, \boldsymbol{\Lambda}) = \begin{bmatrix} \boldsymbol{N}(\boldsymbol{U}) + \xi \mathcal{K} \boldsymbol{U} - \boldsymbol{F} + B_{\text{id}}^T \boldsymbol{\Lambda} \\ B_{\text{id}} \boldsymbol{U} - \boldsymbol{G}_{\text{bc}} \end{bmatrix}. \tag{1.28}$$

Our goal is to drive this residual to 0.

1.2.1. Infinite-dimensional Newton iteration. The first approach falls under the family of infinite-dimensional newton steppers. That is, we start with an initial guess, typically a solution to the Poisson problem $\nabla^2 u = 1/f$ with the same boundary and inter-element conditions. Then, we seek a descent direction $\delta \begin{bmatrix} \boldsymbol{U} \\ \boldsymbol{\Lambda} \end{bmatrix}$ which will reduce the residual $\boldsymbol{R}(\boldsymbol{U}, \Lambda)$, update the current solution, and iterate until we achieve some tolerance for the residual. If we view the weak form of (1.21) as corresponding to the first variation of an energy functional $\mathcal{L}$ (whether it does in the right sense is unclear - this is an inverse problem of calculus of variations), then driving the residual to 0 is akin to satisfying the necessary condition for a minimizer of the functional ($\delta\mathcal{L}(u, v) = 0$). Naturally, a strong descent direction can be found by quadratic approximation of the functional in perturbations of u by εw belonging to the same space as v, which requires its second variation, or, in other words, the first variation of the weak form. For triangle T_k, this is

$$\mathcal{H}_{ij}(\boldsymbol{u}^k) = \int_{T_{\text{ref}}} \phi_j |A_k| \big(A_k^{-1}\nabla\phi_i\big) \cdot \frac{\Big(\sum_{l=1}^{M} u_l^k \big(A_k^{-1}\nabla\phi_l\big)\Big)}{\sum_{l=1}^{M}\sum_{p=1}^{M} u_l^k u_p^k \big(A_k^{-1}\nabla\phi_l\big)\cdot\big(A_k^{-1}\nabla\phi_p\big)} d\boldsymbol{r} + \xi\mathcal{K}_{ij} \tag{1.29}$$

Then, a newton iteration which respects the boundary and inter-element continuity conditions at every step and terminates when the residual meets some tolerance `tol` is:

(i) Initialize a solution $\begin{bmatrix} \boldsymbol{U}^{(0)} \\ \boldsymbol{\Lambda}^{(0)} \end{bmatrix}$ by solving the discretized Poisson problem with constraints

$$\begin{bmatrix} \mathcal{K} & B_{\text{id}}^T \\ B_{\text{id}} & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{U}^{(0)} \\ \boldsymbol{\Lambda}^{(\mathbf{0})} \end{bmatrix} = \begin{bmatrix} \boldsymbol{F} \\ \boldsymbol{G}_{\text{bc}} \end{bmatrix}.$$

(ii) Let $\begin{bmatrix} \boldsymbol{U} \\ \boldsymbol{\Lambda} \end{bmatrix} = \begin{bmatrix} \boldsymbol{U}^{(0)} \\ \boldsymbol{\Lambda}^{(0)} \end{bmatrix}$.

(iii) If $||\boldsymbol{R}(\boldsymbol{U}, \Lambda)|| >$ `tol`, solve for descent direction $\begin{bmatrix} \delta\boldsymbol{U} \\ \delta\boldsymbol{\Lambda} \end{bmatrix}$ through

$$\begin{bmatrix} \mathcal{H}^T & B_{\text{id}}^T \\ B_{\text{id}} & 0 \end{bmatrix} \begin{bmatrix} \delta\boldsymbol{U} \\ \delta\boldsymbol{\Lambda} \end{bmatrix} = -\boldsymbol{R}(\boldsymbol{U}, \boldsymbol{\Lambda})$$

The constraints ensure that the computed descent direction still lies in the correct, constrained solution space.

(iv) Update the solution with an optional step length $0 < \alpha < 1$.

$$\begin{bmatrix} \boldsymbol{U} \\ \boldsymbol{\Lambda} \end{bmatrix} \to \begin{bmatrix} \boldsymbol{U} \\ \boldsymbol{\Lambda} \end{bmatrix} + \alpha \begin{bmatrix} \delta\boldsymbol{U} \\ \delta\boldsymbol{\Lambda} \end{bmatrix}$$

(v) Repeat from step (iii) until the residual norm is less than `tol`.

This approach works for viscosity parameter $\xi \sim \mathcal{O}(1e2)$, but is rather unstable for smaller ξ. This holds true even if we use homotopic continuation methods, where we have a loop wrapping around the Newton method which starts with $\xi = 0.1$, decreases ξ when the residual tolerance is met, and repeats with the solution for the previous ξ as the initial guess for the Newton iteration. I found that as $\xi \to 0$, the magnitude of solution modes across the spectrum "lifts" up. This means that $\xi\nabla^2 u$ serves as a smoothing operator on high frequency solution components, and decreasing ξ too much results in a degenerate weak form. The Newton step will have many null components in modal directions that don't affect the residual, and the new direction drifts into moving the higher-energy non-physical directions (higher degree modes). This takes us away from the correct viscosity limit.

1.2.2. Pseudo-time relaxation with an IMEX method. The second approach is one in which we reduce the residual by casting the problem as a system of first-order ordinary differential algebraic equations with a fictitious time parameter τ. We write

$$\partial_\tau \boldsymbol{U} = -\boldsymbol{R}(\boldsymbol{U}, \boldsymbol{\Lambda}), \tag{1.30}$$

and discretize this DAE system in time with common techniques. The first we try is an implicit-explicit discretization, where we handle the non-linear term explicitly, and the stiff linear term implicitly. That is,

$$\begin{aligned} \frac{\boldsymbol{U}^{n+1} - \boldsymbol{U}^n}{\Delta\tau} &= -(\boldsymbol{N}(\boldsymbol{U}^{\boldsymbol{n}}) + \xi\mathcal{K}\boldsymbol{U}^{\boldsymbol{n+1}} - \boldsymbol{F}) \\ \Rightarrow (I + \xi\Delta\tau\mathcal{K})\boldsymbol{U}^{n+1} &= \boldsymbol{U}^n - \Delta\tau(\boldsymbol{N}(\boldsymbol{U}^n) - \boldsymbol{F}) \end{aligned} \tag{1.31}$$

We want to enforce the constraints at each time step, so we solve the augmented system

$$\begin{bmatrix} I + \xi\Delta\tau\mathcal{K} & B_{\text{id}}^T \\ B_{\text{id}} & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{U}^{n+1} \\ \boldsymbol{\Lambda}^{n+1} \end{bmatrix} = \begin{bmatrix} \boldsymbol{U}^n - \Delta\tau(\boldsymbol{N}(\boldsymbol{U}^n) - \boldsymbol{F}) \\ \boldsymbol{G}_{\text{bc}} \end{bmatrix} \tag{1.32}$$

So far, this method is the fastest, and most accurate in terms of converging to a reasonable solution for moderate ξ. A plot of the solution is depicted below for the unit square. Note, ξ has the effect of preserving the behavior of the time to boundary, but either reducing or increasing the maximum value (which occurs at the medial point (1/2,1/2).)

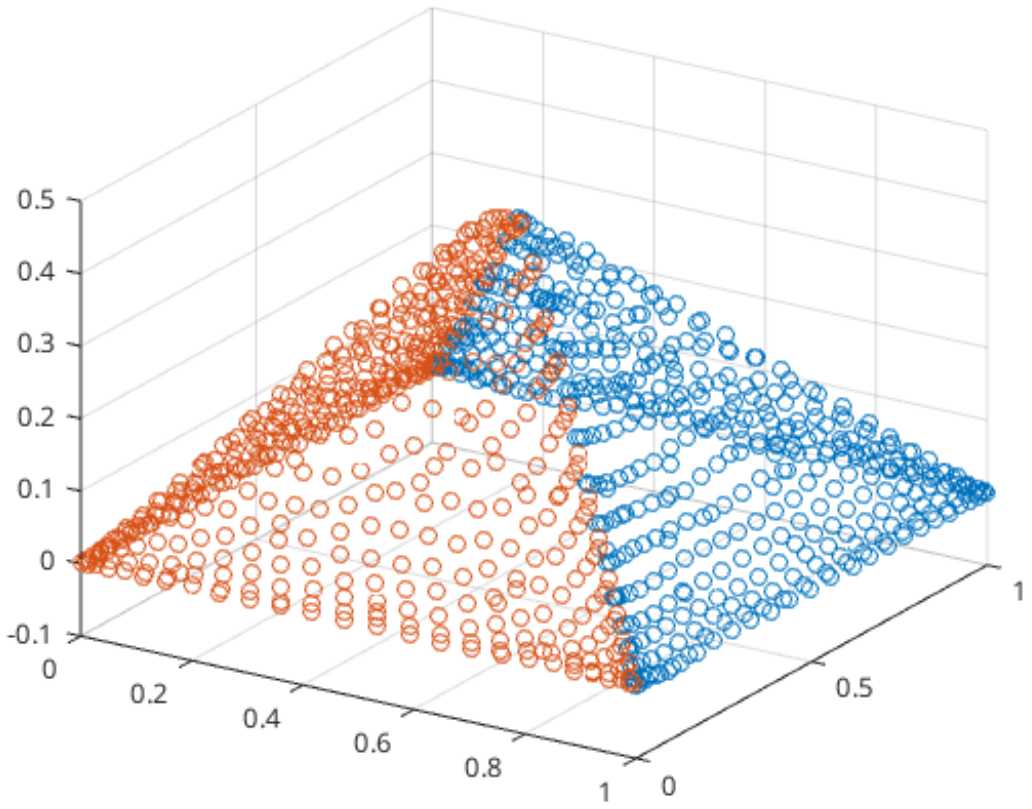

FIG. 5. *Solution to* (1.21) *after 100 timesteps, with* $\Delta\tau = 0.01$, $\xi = 0.01$ *on the unit square* $[0,1]^2$ *discretized with two triangles.*

I would like to work-out the ETDRK2 method for this problem. It is specifically designed to allow larger time steps for stiff problems with a non-linear part, and exhibits second-order accuracy in time.

1.2.3. Pseudo-time relaxation with the ETDRK2 method. We start by describing `ETDRK2` on a scalar semi-linear equation. Consider

$$\frac{du}{dt} = Lu + Nu$$

with $u(\tau^n) = u^n$, L a scalar, and N a nonlinear scalar valued function of u. The update proceeds in two stages, delineated below, where the time step size is $d\tau$.

(i) Compute the exponential step

$$\phi_1 = e^{d\tau L} u^n$$

(ii) Evaluate the nonlinearity

$$N_1 = N(u^n)$$

(iii) Midpoint update

$$\Phi_2 = \phi_1 + d\tau\varphi(d\tau L)N_1$$

(iv) Evaluate nonlinearity at midpoint

$$N_2 = N(\Phi_2)$$

(v) Final update

$$u^{n+1} = \phi_1 + d\tau\varphi_1(d\tau L)N_1 + d\tau\varphi_2(d\tau L)(N_2 - N_1)$$

Above, φ_1, φ_2 are entire functions defined by

$$\varphi_1(z) = \frac{e^z - 1}{z} \tag{1.33}$$

$$\varphi_2(z) = \frac{e^z - 1 - z}{z^2}. \tag{1.34}$$

2. Quadrature generation for Jacobi polynomials on the Simplex. Here, we detail the procedure for generating quadrature rules on the simplex. That is, given $f : \mathcal{T}^d \to \mathbb{R}$, we seek a set of N (d+1)-tuples $(\boldsymbol{X_1}, \cdots, \boldsymbol{X_d}, \boldsymbol{W})$ such that

$$\begin{aligned} &\int_{\mathcal{T}^d} f(x_1, \cdots, x_d) w^{(\boldsymbol{a})}(x_1, \cdots, x_d) dx_1 \cdots dx_d \\ &\approx \sum_{j=1}^{N} f(x_{j1}, \cdots, x_{jd}) w_j, \end{aligned} \tag{2.1}$$

where $w^{(\boldsymbol{a})}(x_1, \cdots, x_d)$ is the normalized Jacobi weight function on $\mathcal{T}^d$, and the approximation is exact for $M > N$ polynomials

Suppose that $\boldsymbol{P}(\boldsymbol{x}) \in \mathbb{R}^N$, with

$$N = \dim(\Pi_{n-1}^d) = \binom{n-1+d}{n-1},$$

as in (1.6) (in which we had $d = 2$), and $\boldsymbol{x} \in \mathbb{R}^d$. The orthonormal 3-term relation for Jacobi polynomials on the simplex is given by

$$x_i \mathbb{P}_n = A_{n,i} \mathbb{P}_{n+1} + B_{n,i} \mathbb{P}_n + A_{n-1,i}^T \mathbb{P}_{n-1}, \ i = 1, \cdots d \tag{2.2}$$

$$A_{n,i} = \langle x_i \mathbb{P}_n, \mathbb{P}_{n+1}^T \rangle \in \mathbb{R}^{r_n^d \times r_{n+1}^d}, \tag{2.3}$$

$$B_{n,i} = \langle x_i \mathbb{P}_n, \mathbb{P}_n^T \rangle \in \mathbb{R}^{r_n^d \times r_n^d}, \tag{2.4}$$

where $\langle \cdot \rangle$ is the weighted L^2 inner product. The matrix recurrence generates an analog of the Jacobi matrix from 1D, except now there are d of them - one for each dimension:

$$\mathcal{J}_{n,i} = \begin{bmatrix} B_{0,i} & A_{0,i} & & & \\ A_{0,i}^T & B_{1,i} & A_{1,i} & & \\ & A_{1,i}^T & \ddots & \ddots & \\ & & \ddots & B_{n-2,i} & A_{n-2,i} \\ & & & A_{n-2,i}^T & B_{n-1,i} \end{bmatrix}, \ i = 1, \cdots d \tag{2.5}$$

with $\mathcal{J}_{n,i} \in \mathbb{R}^{N \times N}$ symmetric. By definition, we see that the Jacobi matrices satisfy

$$x_i \boldsymbol{P} = \mathcal{J}_{n,i} \boldsymbol{P} + \begin{bmatrix} \boldsymbol{0}_{(N - r_{n-1}^d) \times 1} \\ A_{n-1,i} \mathbb{P}_n \end{bmatrix}, \quad i = 1, \cdots, d. \tag{2.6}$$

We hope for a way to generalize the univariate Golub-Welsch algorithm for determining Gaussian quadrature from orthogonal polynomials on the line to the multi-variate case on the simplex. We say that $\Lambda = (\lambda_1, \cdots, \lambda_d)$ is a *joint eigenvalue* of $J_{n,1}, \cdots, J_{n,d}$ if there is a vector $\xi \neq 0$ such that $J_{n,i}\xi = \lambda_i \xi$ for $i = 1, \cdots, d$. The vector ξ is the *joint eigenvector*. If there are N distinct joint eigenvalues, then the weights and abscissa for the Gaussian quadrature are as in 1D - the nodes are the joint eigenvalues, and the weights are the square of the first component of the corresponding joint eigenvector. However, Theorem 3.7.5 of Dunkl-Xu states that a necessary and sufficient condition for this to occur is the following commutativity relation

$$A_{n-1,i} A_{n-1,j}^T = A_{n-1,j} A_{n-1,i}^T, \ 1 \leq i, j \leq d \tag{2.7}$$

This is equivalent to stating that $J_{n,i}$ are a commuting family, and hence, simultaneously diagonalizable. By inspection, we see that the coefficient matrices in the recurrence for Jacobi polynomials on the triangle do not, in general, satisfy the commutativity relation. Thus, *there do not exist minimal Gaussian quadrature rules on the Triangle using the classic orthogonal polynomials.*

We need an accurate quadrature on the triangle, as the convergence of any modal solver depends highly on accurate representations of the right-hand-side in the modal basis. While $J_{n,i}$ may not commute, there is a connection between their spectrums and non-minimal Gaussian-like quadrature rules on $\mathcal{T}^d$. Vioreanu and Rokhlin show that the joint eigenvalues of the combined complex operator

$$\mathcal{J}_n = \mathcal{J}_{n,1} + \iota \mathcal{J}_{n,2} \tag{2.8}$$

lie within $\mathcal{T}^2$ (Theorem 3.4 in VRQUAD). The algorithm described in their work uses the constant unit weight function and gives an alternate way to evaluate $\mathcal{J}_n$. For the case of the weighted space $L^2(\mathcal{T}, w_{a,b,c} W(x,y))$ with normalized weight (see (1.2)), the entries are

$$(\mathcal{J}_n)_{i,j} = \int_{\mathcal{T}} (x_1 + \iota x_2) P_j(x,y) P_i(x,y) w_{a,b,c} W(x,y) dxdy, \quad 0 \leq i, j \leq N-1, \tag{2.9}$$

where P_j is the j^{th} component of $\boldsymbol{P} = (\mathbb{P}_0^T, \mathbb{P}_1^T, \cdots, \mathbb{P}_{n-1}^T)^T = (P_0^0, P_0^1, P_1^1, \cdots P_n^n)^T \in \mathbb{R}^N$. These are precisely the same operators defined in (2.8). However, this "complexification" trick is not available to us in dimensions $d > 2$, but there do exist algorithms for finding *approximate* joint diagonalizations of non-commuting matrix families.

2.1. Approximate joint diagonalization of non-commuting symmetric matrices. The approximate joint diagonalization problem for a set of real-valued symmetric matrices $\{C^1, \cdots, C^d\}$ is to find the orthogonal matrix U such that the matrices

$$F^k = UC^kU^T, \quad 1 \le k \le d \tag{2.10}$$

are approximately diagonal. Alternatively, we can find the diagonal matrices F^k such that C^k are approximately equal to U^TF^kU. The definition of "as diagonal as possible" leads to various formulations of the diagonalization problem in terms of minimizing an objective function. For example, solving

$$\min_{\substack{U \in \mathbb{R}^N, \\ U^TU = UU^T = I}} \sum_{i \neq j} (F^k_{ij})^2 \tag{2.11}$$

is equivalent to minimizing the off-diagonal entries of F^k by orthogonal similarity transformations of C^k. On the other hand, solving

$$\min_{\substack{U, F^1, \cdots F^d, \\ U^TU = UU^T = I}} \sum_{k=1}^{d} \left|\left|C^k - U^TF^kU\right|\right|_F^2 \tag{2.12}$$

is finding the best representation of C^k in the basis of U - a subspace fitting approach. For both problems, we require orthogonality of U, as we want it to represent the "average eigenstructure" of C^k. For the Jacobi matrices $J_{n,i}$, this requirement is natural. We want a decomposition close to the optimal quadrature case of exact joint diagonlization. For this purpose, we use the algorithm specified in "Jacobi Angles for Simultaneous Diagonalization". The joint-eigenvalues provide us with an initial guess to a subsequent optimization problem.

2.2. Quadrature via roots. We want to integrate the M polynomials up to total degree $m-1$ exactly with $m \ge n$ using only N nodes and weights. That is, with $M = \dim(\Pi^d_{m-1}) = \binom{m-1+d}{m-1}$, the integrals for which we want an exact quadrature are

$$\begin{aligned} \mathcal{I}_j &= \int_{\mathcal{T}^d} P_j(\boldsymbol{x}) w_{\boldsymbol{a}} W(\boldsymbol{x}) d(\boldsymbol{x}), \quad 0 \le j \le M-1 \\ &= \langle P_j, P_0 \rangle = \delta_{j0}, \end{aligned} \tag{2.13}$$

where we use mutual orthonormality under $w_{a,b,c}W(x,y)$ (1.4) to evaluate them and arrive at the δ_{j0} term. If $\boldsymbol{X}_1, \ldots, \boldsymbol{X}_d, \boldsymbol{W} \in \mathbb{R}^N$ are the nodes and weights, $\boldsymbol{\mathcal{I}} \in \mathbb{R}^M$ the above integrals, and $\boldsymbol{P}(\boldsymbol{x}) \in \mathbb{R}^M$ (with $\boldsymbol{x} \in \mathbb{R}^d$) our $M > N$ polynomials, the objective function we aim to find a zero of is $\boldsymbol{F} : \mathbb{R}^{N \times (d+1)} \to \mathbb{R}^M$ given by

$$\boldsymbol{F}(\boldsymbol{X}_1, \cdots, \boldsymbol{X}_d, \boldsymbol{W}) = \boldsymbol{P}(\tilde{\boldsymbol{z}})\boldsymbol{W} - \boldsymbol{e}_1, \quad \boldsymbol{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \in \mathbb{R}^M, \ \boldsymbol{P}(\tilde{\boldsymbol{z}}) \in \mathbb{R}^{M \times N}, \tag{2.14}$$

where

$$\tilde{\boldsymbol{z}} = \begin{bmatrix} \boldsymbol{X}_1 & \cdots & \boldsymbol{X}_d \end{bmatrix} \in \mathbb{R}^{N \times d} \tag{2.15}$$

Now, letting

$$\boldsymbol{z} = \begin{bmatrix} \tilde{\boldsymbol{z}} & \boldsymbol{W} \end{bmatrix} \in \mathbb{R}^{N \times (d+1)}, \tag{2.16}$$

we can use the Gauss-Newton iteration (where A^+ is the Moore-Penrose pseudoinverse)

$$\boldsymbol{z}^{k+1} = \boldsymbol{z}^k - \alpha_k [\nabla(\boldsymbol{F}(\boldsymbol{z}^k))]^+ \boldsymbol{F}(\boldsymbol{z}^k), \tag{2.17}$$

to find roots of (2.14), where $\alpha \in (0,1)$. As we show in Section **??**, we have access to analytical formula for computing the derivatives of (2.14), which enables us to alternatively use the damped Newton iteration

$$\boldsymbol{z}^{k+1} = \boldsymbol{z}^k - \alpha [\mathcal{D}^2(\boldsymbol{F}(\boldsymbol{z}^{\boldsymbol{k}}))]^{-1} \nabla(\boldsymbol{F}(\boldsymbol{z}^k)). \tag{2.18}$$

This iteration does not work well, as the Hessian for each component of the objective is not well conditioned, i.e. near singular. Using the convex problem is a better approach. However, the Gauss-Newton iteration gives rapid convergence towards the solution, albeit with violation of the constraints. It might be wise to use Gauss-Newton to further initialize the guess for the solution, before passing it to iteration (2.20). We also consider the Newton iteration for the equivalent problem of finding roots of the function $f : \mathbb{R}^{N \times (d+1)} \to \mathbb{R}$ defined by

$$f(\boldsymbol{z}) := \frac{1}{2} ||\boldsymbol{F}(\boldsymbol{z})||_2^2, \tag{2.19}$$

with iteration

$$\boldsymbol{z}^{k+1} = \boldsymbol{z}^k - \alpha [\mathcal{D}^2(f(\boldsymbol{z}^k))]^{-1} \nabla(f(\boldsymbol{z}^k)) \tag{2.20}$$

The choice of initial point $\boldsymbol{z}^0 = (\tilde{\boldsymbol{z}}^0, \boldsymbol{W}_0)$ and control of the step length α_k are vital for its convergence. Moreover, we'd like to incorporate inequality constraints on $\boldsymbol{z}^k$ to enforce that quadrature nodes are interior to $\mathcal{T}^2$. Positivity of weights is also desirable, though not absolutely necessary for stability when the absolute value of each weight is not large. By casting the problem of finding roots of (2.14), (2.19) into a minimization problem , we can use techniques from constrained optimization to find acceptable quadratures.

First, we make a few observations about the objective function (2.19). Expanding out the norm in terms of the partitioning of the variable $\boldsymbol{z}$ defined in (2.16), we have

$$f(\boldsymbol{z}) = \frac{1}{2}\boldsymbol{W}^T\boldsymbol{P}(\tilde{\boldsymbol{z}})^T\boldsymbol{P}(\tilde{\boldsymbol{z}})\boldsymbol{W} - (\mathbf{1}_{N\times 1}^T\boldsymbol{W}) + \frac{1}{2}. \tag{2.21}$$

Since $M > N$, and each of the M polynomials is independent, $\boldsymbol{P}(\tilde{\boldsymbol{z}})$ has full column rank for any $\tilde{\boldsymbol{z}} \in \mathbb{R}^{N\times d}$ comprised of N distinct points in $\mathcal{T}^d \subset \mathbb{R}^d$. This implies that $\boldsymbol{P}(\tilde{\boldsymbol{z}})^T\boldsymbol{P}(\tilde{\boldsymbol{z}})$ is positive definite, and (2.21) is a quadratic, strongly convex function in $\boldsymbol{W}$. The optimal $\boldsymbol{W}$ is easily determined given the necessary and sufficient optimality condition for strongly convex functions; namely, $\nabla_{\boldsymbol{W}} f(\boldsymbol{z}) = 0$. Hence,

$$\begin{aligned} \nabla_{\boldsymbol{W}} f(\boldsymbol{z}) &= \boldsymbol{P}(\tilde{\boldsymbol{z}})^T\boldsymbol{P}(\tilde{\boldsymbol{z}})\boldsymbol{W} - \mathbf{1}_{N\times 1} = 0 \\ &\Rightarrow \boldsymbol{W} = [\boldsymbol{P}(\tilde{\boldsymbol{z}})^T(\boldsymbol{P}(\tilde{\boldsymbol{z}}))]^{-1}\mathbf{1}_{N\times 1}. \end{aligned} \tag{2.22}$$

Inserting (2.22) into (2.14) gives

$$\begin{aligned} \tilde{\boldsymbol{F}}(\tilde{\boldsymbol{z}})^T &= \mathbf{1}_{1\times N}[\boldsymbol{P}(\tilde{\boldsymbol{z}})^T\boldsymbol{P}(\tilde{\boldsymbol{z}})]^{-1}\boldsymbol{P}(\tilde{\boldsymbol{z}})^T - \boldsymbol{e}_1^T \\ \Rightarrow \boldsymbol{F}(\tilde{\boldsymbol{z}}) &= \boldsymbol{P}(\tilde{\boldsymbol{z}})^\dagger\mathbf{1}_{N\times 1} - \boldsymbol{e}_1 \end{aligned}$$

The condition number of $\boldsymbol{P}(\tilde{\boldsymbol{z}})^T\boldsymbol{P}(\tilde{\boldsymbol{z}})$ grows as the polynomial order increases. Hence, we will need some kind of preconditioning before computing the weights. In particular, the pre-conditioning can be a derivative-free optimization on the condition number of $\boldsymbol{P}^T(\tilde{\boldsymbol{z}})$ for a fixed W, ensuring the value of $f(\boldsymbol{z})$ does not exceed its initial value. I've already implemented this with NLOPT, and it works well. .

2.2.1. Roots via minimization. The optimization problem we would like to solve, stated in canonical form, is

$$\begin{aligned} &\min_{\boldsymbol{z}\in\mathbb{R}^{N\times(d+1)}} f(\boldsymbol{z}) \quad \text{such that} \\ &\qquad f_i(\boldsymbol{z}) \le 0, \quad i = 1, \cdots (d+1)N \end{aligned} \tag{2.23}$$

where $\boldsymbol{f} : \mathbb{R}^{N\times(d+1)} \to \mathbb{R}^{(d+1)N} := \begin{bmatrix} f_1(\boldsymbol{z}) \\ \vdots \\ f_{(d+1)N}(\boldsymbol{z}) \end{bmatrix}$ is given by

$$\boldsymbol{f}(\boldsymbol{z}) = G\cdot\mathtt{vec}(\boldsymbol{z}) - \begin{bmatrix} \mathbf{0_{dN\times 1}} \\ \mathbf{1_{N\times 1}} \end{bmatrix}, \quad G = \begin{bmatrix} -I_1 & \times & \cdots & \times & \times \\ \times & \ddots & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ \times & \cdots & \cdots & -I_d & \times \\ I_1 & \cdots & \cdots & I_d & \times \end{bmatrix} \in \mathbb{R}^{(d+1)N\times(d+1)N}, \tag{2.24}$$

and f is given by (2.19). The zero entries in the constraint matrix G are denoted by $\times$, and each $I_1, \cdots, I_d$ is an $N \times N$ identity matrix. Specifically, each block row of the constraint equation $\boldsymbol{f}(\boldsymbol{z})$ corresponds, respectively, to the following inequality constraints for $j = 1 \cdots N$:

(i) $X_{1j} \ge 0, \cdots, X_{dj} \ge 0$, which restricts the solution to the non-negative dN-orthant .
(ii) $|\boldsymbol{X}_j|_1 \le 1$, which is the L_1-norm definition of the simplex $\mathcal{T}^d$, and means that no abscissa points should be outside of it.

Now, we can optimize over specific partitions of the variable $\boldsymbol{z}$ independently, and formulate an equivalent problem. With $\tilde{\boldsymbol{z}}$ as in (2.15), define $\tilde{f} : \mathbb{R}^{N\times d} \to \mathbb{R}$ by

$$\begin{aligned} \tilde{f}(\tilde{\boldsymbol{z}}) &= \inf\{f(\begin{bmatrix}\tilde{\boldsymbol{z}} & \boldsymbol{W}\end{bmatrix}) \mid \boldsymbol{W} \in \mathbb{R}^N\} \\ &= f(\tilde{\boldsymbol{z}}, [\boldsymbol{P}(\tilde{\boldsymbol{z}})^T\boldsymbol{P}(\tilde{\boldsymbol{z}})]^{-1}\mathbf{1}_{N\times 1}) \\ &= \frac{1}{2}(\mathbf{1}_{1\times N}[\boldsymbol{P}(\tilde{\boldsymbol{z}})^T\boldsymbol{P}(\tilde{\boldsymbol{z}})]^{-1}\mathbf{1}_{N\times 1} - 2(\mathbf{1}_{1\times N}[\boldsymbol{P}(\tilde{\boldsymbol{z}})^T\boldsymbol{P}(\tilde{\boldsymbol{z}})]^{-1}\mathbf{1}_{N\times 1}) + 1) \\ &= -\frac{1}{2}(\mathbf{1}_{1\times N}[\boldsymbol{P}(\tilde{\boldsymbol{z}})^T\boldsymbol{P}(\tilde{\boldsymbol{z}})]^{-1}\mathbf{1}_{N\times 1} - 1) \end{aligned} \tag{2.25}$$

where we inserted the optimal $\boldsymbol{W}$ found in (2.22). The problem (2.23) is then equivalent to

$$\begin{aligned} &\min_{\tilde{\boldsymbol{z}}\in\mathbb{R}^{N\times d}} \tilde{f}(\tilde{\boldsymbol{z}}) \quad \text{such that} \\ &\qquad \tilde{f}_i(\tilde{\boldsymbol{z}}) \le 0, \quad i = 1, \cdots, (d+1)N \end{aligned} \tag{2.26}$$

where $\tilde{\boldsymbol{f}} : \mathbb{R}^{N\times d} \to \mathbb{R}^{(d+1)N} := \begin{bmatrix} \tilde{f}_1(\tilde{\boldsymbol{z}}) \\ \vdots \\ \tilde{f}_{(d+1)N}(\tilde{\boldsymbol{z}}) \end{bmatrix}$ is given by

$$\tilde{\boldsymbol{f}}(\tilde{\boldsymbol{z}}) = G \cdot \mathtt{vec}(\tilde{\boldsymbol{z}}) - \begin{bmatrix} \mathbf{0_{dN\times 1}} \\ \mathbf{1_{N\times 1}} \end{bmatrix}, \quad G = \begin{bmatrix} -I_1 & \times & \cdots & \times \\ \times & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ \times & \cdots & \cdots & -I_d \\ I_1 & \cdots & \cdots & I_d \end{bmatrix} \in \mathbb{R}^{(d+1)N\times dN}, \tag{2.27}$$

Let $L : \mathbb{R}^{dN} \times \mathbb{R}^{(d+1)N} \to \mathbb{R}$ denote the *Lagrangian* of the *primal objective* (2.26). With *dual variables* $\boldsymbol{\lambda} \in \mathbb{R}^{(d+1)N}$, it is given by

$$L(\tilde{\boldsymbol{z}}, \lambda) = \tilde{f}(\tilde{\boldsymbol{z}}) + \boldsymbol{\lambda}^T \tilde{\boldsymbol{f}}(\tilde{\boldsymbol{z}}). \tag{2.28}$$

The *Lagrange dual function* of (2.28) is defined as the minimum value of the Lagrangian over $\boldsymbol{z}$:

$$g(\boldsymbol{\lambda}) = \inf_{\tilde{\boldsymbol{z}}\in\mathbb{R}^{(dN}} L(\tilde{\boldsymbol{z}}, \boldsymbol{\lambda}). \tag{2.29}$$

The dual function gives a lower bound on the optimal value $p^\star$ of the optimization problem (2.26). It stands to reason that there may exist a "best" lower bound, obtainable through solving the *Lagrange dual problem*:

$$\begin{aligned} &\max_{\boldsymbol{\lambda}\in\mathbb{R}^{(d+1)N}} g(\boldsymbol{\lambda}) \\ &\text{subject to } \boldsymbol{\lambda} \succeq 0 \end{aligned} \tag{2.30}$$

The dual problem is a concave optimization problem, since the objective $g(\boldsymbol{\lambda})$ is the pointwise infimum of a family of affine functions of $\boldsymbol{\lambda}$ (they are the sums of hyperplanes with normal $\boldsymbol{\lambda}$). This holds regardless of the convexity of the original objective $\tilde{f}(\tilde{\boldsymbol{z}})$. Thus, there exists a notion of the *duality gap*. If $p^\star$ is the optimal value of the primal problem (2.26), and $d^\star$ is that of the dual problem (2.30), then the lower bound property of the dual problem implies, regardless of the convexity of the primal objective, that

$$d^\star \leq p^\star. \tag{2.31}$$

The difference $p^\star - d^\star$ is referred to as the duality gap. If the duality gap is zero, then we say that *strong duality* holds. That is, the bound obtained from the Lagrange dual function is tight, and solving the dual problem gives us a solution for the primal problem. The dual problem can be solved using the regular techniques of constrained convex optimization, though we have described it only for theoretical clarification.

If $\tilde{\boldsymbol{z}}^\star$ and $(\boldsymbol{\lambda}^\star)$ are primal and dual optimal points, respectively, with zero duality gap, then $\tilde{\boldsymbol{z}}^\star$ minimizes $L(\tilde{\boldsymbol{z}}, \boldsymbol{\lambda}^\star)$ over $\tilde{\boldsymbol{z}}$. Hence, the first order necessary optimality conditions must hold; namely, that the gradient of L must vanish:

$$\nabla \tilde{f}(\tilde{\boldsymbol{z}}^\star) + \boldsymbol{\lambda}^{\star T} \nabla \tilde{\boldsymbol{f}}(\tilde{\boldsymbol{z}}^\star) = 0. \tag{2.32}$$

This leads to the following conditions characterizing the solution to any optimization problem for which strong duality holds, known as the Karush-Kuhn-Tucker (KKT) conditions, which we specify for (2.23):

$$\begin{aligned} \tilde{\boldsymbol{f}}(\tilde{\boldsymbol{z}}^\star) &\preceq 0, \\ \boldsymbol{\lambda}^\star &\succeq 0, \\ \boldsymbol{\lambda}^{\star T} \tilde{\boldsymbol{f}}(\tilde{\boldsymbol{z}}^\star) &= 0, \\ \nabla \tilde{f}(\tilde{\boldsymbol{z}}^\star) + \boldsymbol{\lambda}^{\star T} \nabla \tilde{\boldsymbol{f}}(\tilde{\boldsymbol{z}}^\star) &= 0. \end{aligned} \tag{2.33}$$

Remark 2.1. If an optimal primal dual pair is found, $\boldsymbol{z}^\star$ is not necessarily a zero of $\boldsymbol{F}(\boldsymbol{z})$. We have only satisfied the necessary KKT conditions. However, we may be closer to an actual root and will have maintained interior nodes and positive weights. We can use this new estimate $\boldsymbol{z}^\star$ as an initial point in a Gauss-Newton iteration that may relax $\boldsymbol{F}(\boldsymbol{z})$ to zero quadratically.

We will recast (2.23) into a fully unconstrained problem using the barrier method, as described in the next section.

2.2.2. Barrier method. In the barrier method, we start by rewriting (2.26) in a form which incorporates the inequality constraints directly into the objective, as in

$$\min_{\tilde{\boldsymbol{z}}\in\mathbb{R}^{dN}} \tilde{f}(\tilde{\boldsymbol{z}}) + \sum_{i=1}^{(d+1)N} I_-(\tilde{f}_i(\tilde{\boldsymbol{z}})) \tag{2.34}$$

where $I_- : \mathbb{R} \to \mathbb{R}$ is the indicator function for the non-positive reals;

$$I_-(x) = \begin{cases} 0, & x \leq 0 \\ \infty, & x > 0 \end{cases}. \tag{2.35}$$

While (2.34) has no constraints, the objective is not differentiable. As such, we must approximate the indicator function with a sequence of differentiable functions converging to it in the pointwise limit:

$$\tilde{I}_-(x) = -(1/t)\log(-x), \quad t > 0, \quad x \in \mathbb{R}^-_\star. \tag{2.36}$$

Note, we use the convention that $\tilde{I}_-(x > 0) = \infty$. Observe that $\tilde{I}_-(x) \to \infty$ as $x \to 0^-$. As $t \to \infty$ with $x < 0$, $\tilde{I}_-(x) \to 0$. The problem is now approximated with a differentiable objective:

$$\min_{\tilde{\boldsymbol{z}} \in \mathbb{R}^{dN}} t\tilde{f}(\tilde{\boldsymbol{z}}) + \phi(\tilde{\boldsymbol{z}}) \tag{2.37}$$

for $t > 0$, where the log barrier function ϕ is given by

$$\phi(\tilde{\boldsymbol{z}}) = -\sum_{i=1}^{(d+1)N} \log(-\tilde{f}_i(\tilde{\boldsymbol{z}})). \tag{2.38}$$

We can solve (2.37) for increasing values of t, and use the solution from each previous value of t as the initialization for the problem with the next t value. This generates a sequence $\tilde{\boldsymbol{z}}^\star(t)$, known as the *central path*, which converges to the optimal point $\tilde{\boldsymbol{z}}^\star$ as $t \to \infty$ (if the problem is solvable). To each approximate problem for a given t, we can associate the Newton system to solve for the decrement $\Delta\tilde{\boldsymbol{z}}$:

$$(t\nabla^2\tilde{f}(\tilde{\boldsymbol{z}}) + \nabla^2\phi(\tilde{\boldsymbol{z}}))\Delta\tilde{\boldsymbol{z}} = -\big(t\nabla\tilde{f}(\tilde{\boldsymbol{z}}) + \nabla\phi(\tilde{\boldsymbol{z}})\big) \tag{2.39}$$

We provide the relevant derivatives here. First, we define $U : \mathbb{R}^{N\times d} \to \mathbb{R}^{N\times N}$ and $g : \mathbb{R}^{N\times N} \to \mathbb{R}$ by

$$g(U) = -\mathbf{1}_{N\times 1}^T U^{-1}\mathbf{1}_{N\times 1}, \quad U = \boldsymbol{P}(\tilde{\boldsymbol{z}})^T\boldsymbol{P}(\tilde{\boldsymbol{z}}), \quad \tilde{z} = \begin{bmatrix} z_{11} & \cdots & z_{1d} \\ \vdots & \ddots & \vdots \\ z_{N1} & \cdots & z_{Nd} \end{bmatrix}$$

We have the chain rule identity

$$\frac{\partial g(U)}{\partial z_{ij}} = \mathrm{Tr}\Big[\Big(\frac{\partial g(U)}{\partial U}\Big)^T \frac{\partial U}{\partial z_{ij}}\Big]. \tag{2.40}$$

We also know that

$$\frac{\partial g(U)}{\partial U} = U^{-T}\mathbf{1}_{N\times 1}\mathbf{1}_{N\times 1}^T U^{-T}. \tag{2.41}$$

Altogether, we have

$$\begin{aligned}
U_{kl} &= \sum_{q=0}^{M-1} P_q(z_{k1},\dots z_{kd})P_q(z_{l1},\dots,z_{ld}) \\
\frac{\partial U_{kl}}{\partial z_{ij}} &= \sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\partial_{z_{ij}}P_q(z_{l1},\dots,z_{ld}) + P_q(z_{l1},\dots,z_{ld})\partial_{z_{ij}}P_q(z_{k1},\dots,z_{kd}) \\
&= \sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\partial_{z_{ij}}P_q(z_{l1},\dots,z_{ld})\delta_{il} + P_q(z_{l1},\dots,z_{ld})\partial_{z_{ij}}P_q(z_{k1},\dots,z_{kd})\delta_{ik} \\
\Rightarrow \frac{\partial U_{kl}}{\partial z_{ij}} &= \begin{cases} 0, & (i\neq l)\wedge(i\neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\partial_{z_{lj}}P_q(z_{l1},\dots,z_{ld}), & (i=l)\wedge(i\neq k) \\ \sum_{q=0}^{M-1} P_q(z_{l1},\dots,z_{ld})\partial_{z_{kj}}P_q(z_{k1},\dots,z_{kd}), & (i\neq l)\wedge(i=k) \\ 2\sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\partial_{z_{kj}}P_q(z_{k1},\dots,z_{kd}), & (i=l)\wedge(i=k) \end{cases} \\
&= \begin{cases} 0, & (i\neq l)\wedge(i\neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\boldsymbol{P}^j(z_{l1},\dots,z_{ld})^T\mathcal{D}_j\boldsymbol{e}_q, & (i=l)\wedge(i\neq k) \\ \sum_{q=0}^{M-1} P_q(z_{l1},\dots,z_{ld})\boldsymbol{P}^j(z_{k1},\dots,z_{kd})^T\mathcal{D}_j\boldsymbol{e}_q, & (i\neq l)\wedge(i=k) \\ 2\sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\boldsymbol{P}^j(z_{k1},\dots,z_{kd})^T\mathcal{D}_j\boldsymbol{e}_q, & (i=l)\wedge(i=k) \end{cases} \\
&= \begin{cases} 0, & (i\neq l)\wedge(i\neq k) \\ \boldsymbol{P}(z_{k1},\dots,z_{kd})^T\Big(\mathcal{D}_j^T\boldsymbol{P}^j(z_{l1},\dots,z_{ld})\Big), & (i=l)\wedge(i\neq k) \\ \boldsymbol{P}(z_{l1},\dots,z_{ld})^T\Big(\mathcal{D}_j^T\boldsymbol{P}^j(z_{k1},\dots,z_{kd})\Big), & (i\neq l)\wedge(i=k) \\ 2\boldsymbol{P}(z_{k1},\dots,z_{kd})^T\Big(\mathcal{D}_j^T\boldsymbol{P}^j(z_{k1},\dots,z_{kd})\Big), & (i=l)\wedge(i=k) \end{cases}
\end{aligned}$$

For the second derivatives, define $V : \mathbb{R}^{N\times d} \to \mathbb{R}^{N\times N}$ and $h : \mathbb{R}^{N\times N} \to \mathbb{R}^{N\times N}$ by

$$h(V) = V\mathbf{1}_{N\times 1}\mathbf{1}_{N\times 1}^T V, \quad V = U^{-1} = [\boldsymbol{P}(\tilde{\boldsymbol{z}})^T \boldsymbol{P}(\tilde{\boldsymbol{z}})]^{-1}$$

The differential trace identity gives

$$\partial \mathrm{Tr}[h(V)\partial U] = \mathrm{Tr}[\partial h(V)\partial U + h(V)\partial^2 U]$$

So,

$$\frac{\partial \mathrm{Tr}(h(V)\frac{\partial U}{\partial z_{ij}})}{\partial z_{st}} = \mathrm{Tr}\Big[\frac{\partial h(V)}{\partial z_{st}}\frac{\partial U}{\partial z_{ij}} + h(V)\frac{\partial^2 U}{\partial z_{st}\partial z_{ij}}\Big]$$

Now, as before for g, we find for h through the chain rule identity that

$$\frac{\partial h(V)_{kl}}{\partial z_{st}} = \mathrm{Tr}\Big[\Big(\frac{\partial h(V)_{kl}}{\partial V}\Big)^T \frac{\partial V}{\partial z_{st}}\Big] \tag{2.42}$$

Since $V = V^T$, we know that

$$\begin{aligned}\frac{\partial h(V)}{\partial V} &:= \frac{\partial (V^T\mathbf{1}_{N\times 1}\mathbf{1}_{N\times 1}^T V)_{kl}}{\partial V_{ij}} \\ &= \delta_{lj}(V\mathbf{1}_{N\times 1}\mathbf{1}_{N\times 1}^T)_{ki} + \delta_{kj}(\mathbf{1}_{N\times 1}\mathbf{1}_{N\times 1}^T V)_{il}.\end{aligned} \tag{2.43}$$

We can also compute[3]

$$\frac{\partial V}{\partial z_{st}} = \frac{\partial (U^{-1})}{\partial z_{st}} = -U^{-1}\frac{\partial U}{\partial z_{st}}U^{-1} \tag{2.44}$$

[3]This can be seen by taking the derivative of $I = U(\tilde{\boldsymbol{z}})U(\tilde{\boldsymbol{z}})^{-1}$, which is 0, and solving for the desired term.

We handle the Hessian of U below:

$$\frac{\partial^2 U_{kl}}{\partial z_{st}\partial z_{ij}} = \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\partial_{z_{st}}\partial_{z_{lj}} P_q(z_{l1},\dots,z_{ld}) + \partial_{z_{st}} P_q(z_{k1},\dots,z_{kd})\partial_{z_{lj}} P_q(z_{l1},\dots,z_{ld}), & (i = l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{l1},\dots,z_{ld})\partial_{z_{st}}\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd}) + \partial_{z_{st}} P_q(z_{l1},\dots,z_{ld})\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd}), & (i \neq l) \wedge (i = k) \\ 2\sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\partial_{z_{st}}\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd}) + \partial_{z_{st}} P_q(z_{k1},\dots,z_{kd})\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd}), & (i = l) \wedge (i = k) \end{cases}$$

$$= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\partial_{z_{st}}\partial_{z_{lj}} P_q(z_{l1},\dots,z_{ld})\delta_{sl} + \partial_{z_{st}} P_q(z_{k1},\dots,z_{kd})\delta_{sk}\partial_{z_{lj}} P_q(z_{l1},\dots,z_{ld}), & (i = l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{l1},\dots,z_{ld})\partial_{z_{st}}\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd})\delta_{sk} + \partial_{z_{st}} P_q(z_{l1},\dots,z_{ld})\delta_{sl}\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd}), & (i \neq l) \wedge (i = k) \\ 2\sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\partial_{z_{st}}\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd})\delta_{sk} + \partial_{z_{st}} P_q(z_{k1},\dots,z_{kd})\delta_{sk}\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd}), & (i = l) \wedge (i = k) \end{cases}$$

$$= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ 0, & (s \neq l) \wedge (s \neq k) \\ 0, & \big((i = l) \wedge (i = k)\big) \wedge (s \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\partial_{z_{lt}}\partial_{z_{lj}} P_q(z_{l1},\dots,z_{ld}), & \big((i = l) \wedge (i \neq k)\big) \wedge \big((s = l) \wedge (s \neq k)\big) \\ \sum_{q=0}^{M-1} \partial_{z_{kt}} P_q(z_{k1},\dots,z_{kd})\partial_{z_{lj}} P_q(z_{l1},\dots,z_{ld}), & \big((i = l) \wedge (i \neq k)\big) \wedge \big((s \neq l) \wedge (s = k)\big) \\ \sum_{q=0}^{M-1} \partial_{z_{lt}} P_q(z_{l1},\dots,z_{ld})\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd}), & \big((i \neq l) \wedge (i = k)\big) \wedge \big((s = l) \wedge (s \neq k)\big) \\ \sum_{q=0}^{M-1} P_q(z_{l1},\dots,z_{ld})\partial_{z_{kt}}\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd}), & \big((i \neq l) \wedge (i = k)\big) \wedge \big((s \neq l) \wedge (s = k)\big) \\ 2\sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\partial_{z_{kt}}\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd}) + & \\ 2\sum_{q=0}^{M-1} \partial_{z_{kt}} P_q(z_{k1},\dots,z_{kd})\partial_{z_{kj}} P_q(z_{k1},\dots,z_{kd}), & \big((i = l) \wedge (i = k)\big) \wedge (s = k) \end{cases}$$

$$= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ 0, & (s \neq l) \wedge (s \neq k) \\ 0, & \big((i = l) \wedge (i = k)\big) \wedge (s \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\boldsymbol{P}^{jt}(z_{l1},\dots,z_{ld})^T \mathcal{D}_{jt}\boldsymbol{e}_q, & \big((i = l) \wedge (i \neq k)\big) \wedge \big((s = l) \wedge (s \neq k)\big) \\ \sum_{q=0}^{M-1} (\boldsymbol{P}^t(z_{k1},\dots z_{kd})^T \mathcal{D}_t\boldsymbol{e}_q)(\boldsymbol{P}^j(z_{l1},\dots,z_{ld})\mathcal{D}_j\boldsymbol{e}_q), & \big((i = l) \wedge (i \neq k)\big) \wedge \big((s \neq l) \wedge (s = k)\big) \\ \sum_{q=0}^{M-1} (\boldsymbol{P}^t(z_{l1},\dots z_{ld})^T \mathcal{D}_t\boldsymbol{e}_q)(\boldsymbol{P}^j(z_{k1},\dots,z_{kd})\mathcal{D}_j\boldsymbol{e}_q), & \big((i \neq l) \wedge (i = k)\big) \wedge \big((s = l) \wedge (s \neq k)\big) \\ \sum_{q=0}^{M-1} P_q(z_{l1},\dots,z_{ld})\boldsymbol{P}^{jt}(z_{k1},\dots,z_{kd})^T \mathcal{D}_{jt}\boldsymbol{e}_q, & \big((i \neq l) \wedge (i = k)\big) \wedge \big((s \neq l) \wedge (s = k)\big) \\ 2\sum_{q=0}^{M-1} P_q(z_{k1},\dots,z_{kd})\boldsymbol{P}^{jt}(z_{k1},\dots,z_{kd})^T \mathcal{D}_{jt}\boldsymbol{e}_q + & \\ 2\sum_{q=0}^{M-1} (\boldsymbol{P}^t(z_{k1},\dots,z_{kd})\mathcal{D}_t\boldsymbol{e}_q)(\boldsymbol{P}^j(z_{k1},\dots,z_{kd})^T\mathcal{D}_j\boldsymbol{e}_q), & \big((i = l) \wedge (i = k)\big) \wedge (s = k) \end{cases}$$

$$= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ 0, & (s \neq l) \wedge (s \neq k) \\ 0, & \big((i = l) \wedge (i = k)\big) \wedge (s \neq k) \\ \boldsymbol{P}(z_{k1},\dots,z_{kd})^T\big(\mathcal{D}_{jt}^T\boldsymbol{P}^{jt}(z_{l1},\dots,z_{ld})\big), & \big((i = l) \wedge (i \neq k)\big) \wedge \big((s = l) \wedge (s \neq k)\big) \\ \big(\mathcal{D}_t^T\boldsymbol{P}^t(z_{k1},\dots z_{kd})\big)^T\big(\mathcal{D}_j^T\boldsymbol{P}^j(z_{l1},\dots,z_{ld})\big), & \big((i = l) \wedge (i \neq k)\big) \wedge \big((s \neq l) \wedge (s = k)\big) \\ \big(\mathcal{D}_t^T\boldsymbol{P}^t(z_{l1},\dots z_{ld})\big)^T\big(\mathcal{D}_j^T\boldsymbol{P}^j(z_{k1},\dots,z_{kd})\big), & \big((i \neq l) \wedge (i = k)\big) \wedge \big((s = l) \wedge (s \neq k)\big) \\ \boldsymbol{P}(z_{l1},\dots,z_{ld})^T\big(\mathcal{D}_{jt}^T\boldsymbol{P}^{jt}(z_{k1},\dots,z_{kd})\big), & \big((i \neq l) \wedge (i = k)\big) \wedge \big((s \neq l) \wedge (s = k)\big) \\ 2\boldsymbol{P}(z_{k1},\dots,z_{kd})^T\big(\mathcal{D}_{jt}^T\boldsymbol{P}^{jt}(z_{k1},\dots,z_{kd})\big) + & \\ 2\big(\mathcal{D}_t^T\boldsymbol{P}^t(z_{k1},\dots,z_{kd})\big)^T\big(\mathcal{D}_j^T\boldsymbol{P}^j(z_{k1},\dots,z_{kd})\big), & \big((i = l) \wedge (i = k)\big) \wedge (s = k) \end{cases}$$

We verify the correctness of our formulae for the derivatives of our objective by comparing with second-order finite difference approximations. In particular, we show in Figure 6 the relative error in the 2-norm between the analytical evaluation and the finite difference approximation. Since the difference scheme is second order, we expect the error to behave like $\mathcal{O}(h^2)$, where h is the stencil width. That is, we should observe that $\log(||\mathcal{D}(\boldsymbol{F}) - \hat{\mathcal{D}}(\boldsymbol{F})||) \approx 2\log(h) + \beta$, where β is the order constant, and $\hat{\mathcal{D}(F)}$ is the finite difference approximation. On a $\log - \log$ scale, this means the slope of the error line (as a function of stencil width h) should be around 2. Indeed, we do observe this relationship in the regime of asymptotic convergence, before round-off error dominates truncation error for the finite difference scheme.

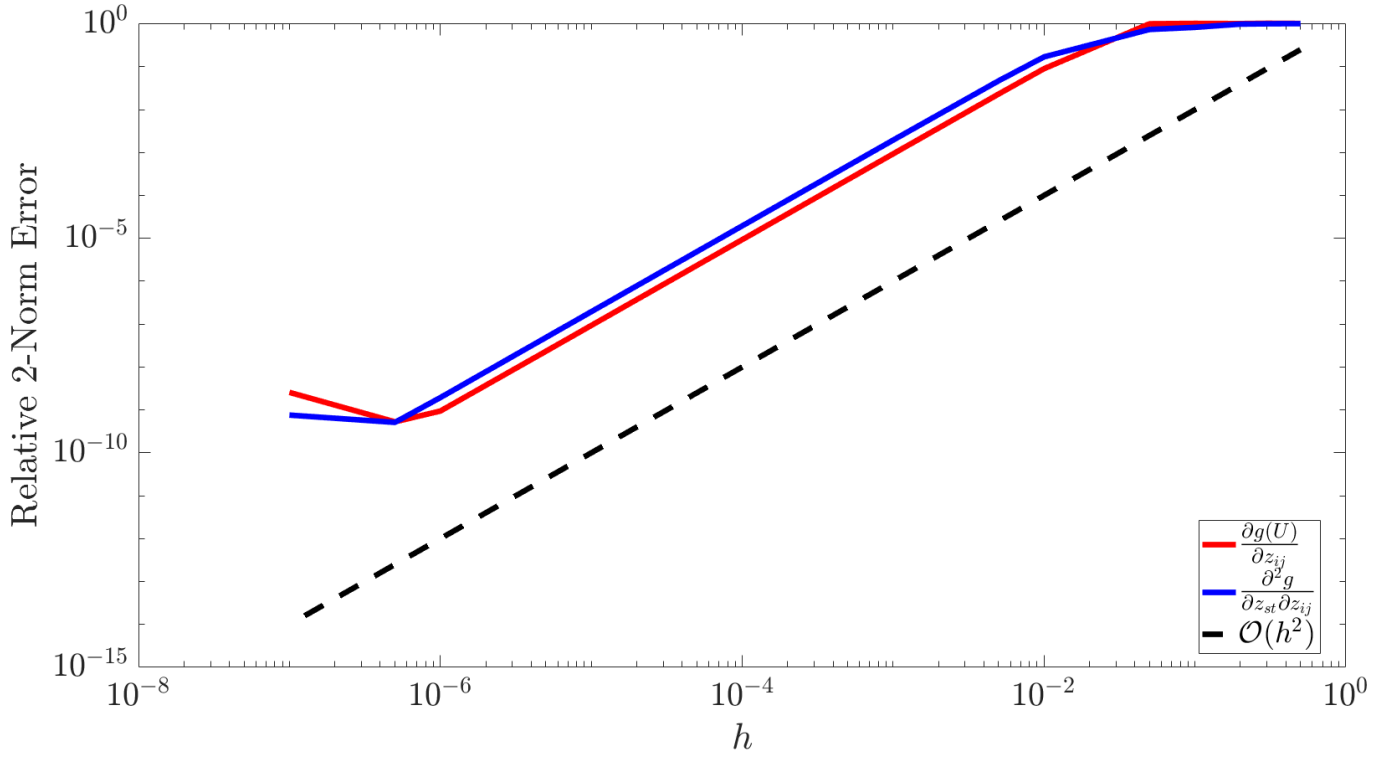

FIG. 6. *Relative error in the 2-norm (Frobenius for matrix valued derivatives) of the analytical derivative formulae when compared to second order finite difference approximations. We see that our formulae must be correct, if our finite difference implementation is correct (which is an initial assumption). The kink in the error curves after which the error increases is typical of a finite difference scheme, and occurs at the "usual" location for at least twice differentiable functons, i.e. $h \approx 1e-6$. This corresponds to the point where numerical round-off error (due to IEEE floating point specification) dominates truncation error.*

Remark 2.2. For this sort of simple "reverse" verification test, we assume that we don't know if our analytical formulae are correct, and that a *verified* finite difference implementation is available. Then, we are essentially re-verifying the finite difference implementation, which on success, certifies correctness of the analytical formulae.

The derivatives of the log-barrier function (2.38) are given by

$$
\begin{aligned}
\nabla\phi(\tilde{\boldsymbol{z}}) &= \sum_{i=1}^{(d+1)N} \frac{-1}{\tilde{f}_i(\tilde{\boldsymbol{z}})}\nabla\tilde{f}_i(\tilde{\boldsymbol{z}}) \\
&= -G^T(\mathbf{1}_{(d+1)N\times 1} \oslash \tilde{\boldsymbol{f}}(\tilde{\boldsymbol{z}})) \\
\nabla^2\phi(\tilde{\boldsymbol{z}}) &= \sum_{i=1}^{(d+1)N} \frac{1}{\tilde{f}_i^2(\tilde{\boldsymbol{z}})}\nabla\tilde{f}_i(\tilde{\boldsymbol{z}})\nabla\tilde{f}_i(\tilde{\boldsymbol{z}})^T - \sum_{i=1} \frac{1}{\tilde{f}_i(\tilde{\boldsymbol{z}})}\nabla^2\tilde{f}_i(\tilde{\boldsymbol{z}}) \\
&= \sum_{i=1}^{(d+1)N} \frac{1}{\tilde{f}_i^2(\tilde{\boldsymbol{z}})}\nabla\tilde{f}_i(\tilde{\boldsymbol{z}})\nabla\tilde{f}_i(\tilde{\boldsymbol{z}})^T \\
&= G^T\left[(\mathbf{1}_{(d+1)N\times 1} \oslash \tilde{\boldsymbol{f}}(\tilde{\boldsymbol{z}}))^{\circ 2} \odot G\right]
\end{aligned}
\tag{2.45}
$$

where we eliminated the second summation in the formula for the hessian of ϕ using the fact that each $\tilde{f}_i$ is linear in $\tilde{\boldsymbol{z}}$. These are verified in Figure 7.

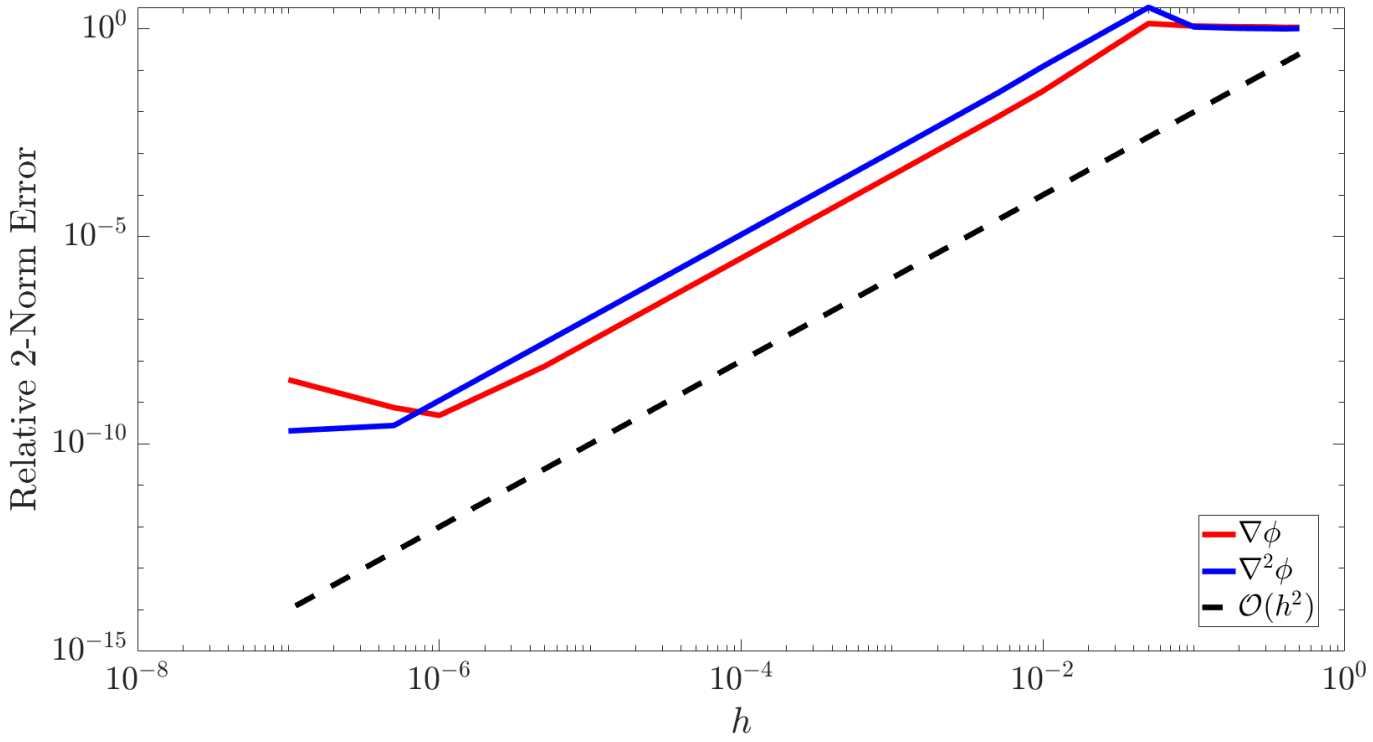

FIG. 7. *Relative error in the 2-norm (Frobenius for matrix valued derivatives) of the analytical derivative formulae for the log-barrier function when compared to second order finite difference approximations.*

TODO: Derive the derivative and promotion operators for the general case of $d > 2$. UPDATE: this is done in the paper "A New Family of Orghogonal Polynomials in Three Variables, by Aktas, Area, and Guldogan" At this point, we can parse out what we mean by applying the inverse of the vector Hessian of $\boldsymbol{F} : \mathbb{R}^{3N} \to \mathbb{R}^M$ to the vector gradient in (2.18). We know that

$$\mathcal{D}^2(\boldsymbol{F}(\boldsymbol{z})) \in \mathbb{R}^{M\times 3N\times 3N}, \quad \mathcal{D}(\boldsymbol{F}(\boldsymbol{z})) \in \mathbb{R}^{M\times 3N}.$$

Then, we take the following natural interpretation[4]

$$\begin{aligned}
[\mathcal{D}^2(\boldsymbol{F}(\boldsymbol{z}))]^{-1} &:= \begin{bmatrix}[\mathcal{D}^2(F_1(\boldsymbol{z}))]^{-1} & \cdots & [\mathcal{D}^2(F_M(\boldsymbol{z}))]^{-1}\end{bmatrix} \\
\nabla(\boldsymbol{F}(\boldsymbol{z})) &:= \begin{bmatrix}\mathcal{D}(F_1(\boldsymbol{z}))^T \\ \vdots \\ \mathcal{D}(F_M(\boldsymbol{z}))^T\end{bmatrix} \\
\Rightarrow [\mathcal{D}^2(\boldsymbol{F}(\boldsymbol{z}))]^{-1}\mathcal{D}(\boldsymbol{F}(\boldsymbol{z})) &:= \sum_{j=1}^{M} \mathcal{D}^2(F_j(\boldsymbol{z}))^{-1}\nabla(F_j(\boldsymbol{z})) \in \mathbb{R}^{3N\times 1}, \quad \text{(as required)}
\end{aligned}$$

3. Image Compression via Jacobi Polynomial Expansion. An image (i.e. with file format .jpg, .png, etc.) can be thought of as an integer-valued surface (or multiple surfaces in the case of multi-channel images) on a rectangular domain. We can easily tesselate this domain with triangles equipped with the quadrature nodes derived in the previous section. The idea is to approximate the subsurfaces on each triangle with a Jacobi polynomial expansion. Areas where the image is highly varying will require more triangles or higher order expansion (i.e. h vs p refinement). As of now, we can compute expansion coefficients on the reference triangle. Here, we first derive a method for computing such expansions on a general triangle. For a given function $f(x, y)$, with $(x, y) \in \mathcal{T}^2$ points in a general triangle

THEOREM 3.1. *Let $X = \{x_1, x_2, \ldots, x_k\}$ be a set of k signed integers, where each x_i is represented using n bits. Let $y = \sum_{i=1}^{k} x_i$ be the sum of all elements in X, which is representable using $m \geq n+1$ bits without overflow (i.e., $-2^{m-1} \leq y \leq 2^{m-1}-1$). Then, there exists an ordering of summation for X that avoids transient overflow when using an m-bit accumulator.*

Proof. We shall proceed by induction.

Base case: Suppose $k = 2$. The set $X = \{x_1, x_2\}$ is comprised of n-bit numbers, and representing the sum $x_1 + x_2$ can require at most $n+1$ bits. Since $m \geq n+1$, the sum does not overflow m-bits. By the commutative property of binary addition, the sum $x_2 + x_1$ will likewise not overflow m-bits. Hence, any ordering of X avoids transient overflow, and so the theorem holds for $k = 2$.

[4]For a function $\boldsymbol{u} : \mathbb{R}^N \to \mathbb{R}^M$, we note the distinction between $\mathcal{D}(\boldsymbol{u}(\boldsymbol{z}))$ and $\nabla(\boldsymbol{u}(\boldsymbol{z}))$. The former is an element of $L(\mathbb{R}^N, \mathbb{R}^M)$, equivalently viewed as a set of M elements of $L(\mathbb{R}^N, \mathbb{R})$, or M row vectors in $\mathbb{R}^N$ (left argument of dot product). The latter evaluated at a point is as an element of $L(\mathbb{R}^M, \mathbb{R}^N)$, equivalently viewed as a set of M elements of $L(\mathbb{R}, \mathbb{R}^N)$, or M column vectors in $\mathbb{R}^N$ (right argument of dot product). TLDR; $[\mathcal{D}(\boldsymbol{u})]^T = \nabla(\boldsymbol{u})$

Inductive hypothesis: Consider an integer $l \geq 2$. Assume that the theorem holds $\forall k \leq l$. That is, there exists an ordering of the set $X = \{x_1, \ldots, x_l\}$ such that the sum of elements of X w.r.t said ordering avoids transient overflow. Denote this ordering by the index set α_l, and the so-ordered set by X_{α_l}.

Inductive step: Suppose that $k = l+1$. Consider the set $X = X_{\alpha_l} \cup x_k$. Assume that the sum of all elements of X is representable by an m-bit signed integer y. We know that

$$y = \sum_{i=1}^{k} x_i = x_k + \sum_{i \in \alpha_l} x_i.$$

By the inductive hypothesis, the second term in the sum avoids transient overflow. Denote this second term by $\hat{y} = \sum_{i \in \alpha_l} x_i$. Since $\hat{y}$ is an m-bit signed integer, and x_k is an n-bit signed integer with $n \leq m-1$, the sum $x_k + \hat{y}$ is represented by at most m bits. Therefore, a feasible ordering to avoid transient overflow is $\alpha_k = \{\alpha_l, k\}$.

Thus, by induction, our theorem must hold for any $k \geq 2$. ∎